

A PROJECT REPORT ON
**Real-Time Collaborative Code
Editor**

Submitted
for the Partial fulfillment of award of
Bachelor of Technology
in
Information Technology
by

Aryan Tyagi (2200270130031)

Ananya Jaiswal (2200270130017)

Ashutosh Singh (2200270130035)

Harsh Kumar (2200270130062)

Under the guidance of
Mr. Nikhil Kumar



**AJAY KUMAR GARG ENGINEERING COLLEGE,
GHAZIABAD**

April 30, 2026

Declaration

We here by declare that the work presented in this report, entitled as **Real-Time Collaborative Code Editor**, was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute. We have given due credit to the original authors and sources for all the words, ideas, diagrams, graphics, computer programs, experiments, and results that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors and sources.

We affirm that no portion of my work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, We shall be fully responsible and answerable.

Name : Aryan Tyagi
Roll No. : 2200270130031

Name : Ananya Jaiswal
Roll No. : 2200270130017

Name : Ashutosh Singh
Roll No. : 2200270130035

Name : Harsh Kumar
Roll No. : 2200270130062

Certificate

This is to certify that the report entitled **Real-Time Collaborative Code Editor** submitted by **Aryan Tyagi (2200270130031)**, **Ananya Jaiswal (2200270130017)**, **Ashutosh Singh (2200270130035)**, **Harsh Kumar (2200270130062)** to the Dr. A. P. J. Abdul Kalam Technical University, Lucknow (U.P.) in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Information Technology is a bonafide record of the project work carried out by him/her under my/our guidance and supervision. This report in any form has not been submitted to any other university or institute for any purpose, to the best of my knowledge.

Mr. Nikhil Kumar

Assistant Professor

Department of Information
Technology

Ajay Kumar Garg

Engineering College

Dr Rahul Sharma

Professor & HOD

Department of Information
Technology

Ajay Kumar Garg

Engineering College

Place: Ghaziabad

April 30, 2026

Acknowledgements

We would like to express our thanks to all the people who have helped bring this project to the stage of fulfillment. We would wish to put on record very special thanks to my major project mentor, **Mr. Nikhil Kumar**, for the support, guidance, encouragement, and some very valuable insight that she guided us in the entire process. Her mentorship has been very pivotal in terms of shaping of our project and leading us toward excellence.

We would like to appreciate our Head of the Department, **Dr. Rahul Sharma**, who had provided us with the wherewithals and put us into an environment that would bring out such innovation towards learning. We would also want to appreciate our teachers and faculty members for all that they share at this crucial juncture in our academic career. We wish to appreciate many more for: helped out or, with their presence, indirectly made contributions to this project.

Contents

Declaration	i
Certificate	ii
Acknowledgements	iii
List of Figures	vi
1 Introduction	1
1.1 Problem Statement of Project	1
1.2 Scope of Project	3
1.3 Detail of Problem Domain	5
1.4 Gantt Chart	8
1.5 System Requirements	9
1.6 Project Report Outline	10
2 Literature Review	11
2.1 Related Study	11
2.2 Research Gaps	17
2.3 Objective of Project	20
3 Methodology Used	22
4 Designing of Project	31
4.1 0-Level DFD	31
4.2 1-Level DFD	33
4.3 2-Level DFD	36
4.4 Use Case Diagram	40
5 Detailed Designing of Project	43
5.1 Class Diagram	43
5.2 Entity-Relationship Diagram (ERD)	47
5.3 Activity Diagram	50
5.4 Sequence Diagram	54

6 Results and Discussion	57
7 Future Work	63
Bibliography	66
Appendix A	68
Appendix B	69
Appendix C	70
Appendix D	72

List of Figures

1.1 Gantt Chart	8
3.1 Flow Diagram of Proposed Realtime Collaborative Code Editor	23
4.1 0-level DFD	31
4.2 1-level DFD	33
4.3 2-level DFD	36
4.4 Use Case Diagram	40
5.1 Class Diagram	43
5.2 ER Diagram	47
5.3 Activity Diagram	50
5.4 Sequence Diagram	54
6.1 Home page	57
6.2 Shared Canvas for Visual Explanation	58
6.3 Code Execution and Output Generation	59
6.4 AI Code Generation using Copilot	60
6.5 Chat Interface for Collaboration	60
6.6 Multi-language Code Editing Interface	61
7.1 Screenshot Of Application	68
7.2 Screenshot Of Application	68
7.3 Research paper	69
7.4 SDG	71
7.5 Plagiarism Report	72
7.6 AI Report	73

Chapter 1

Introduction

1.1 Problem Statement of Project

Programming has become one of the most important skills in the modern digital era. It plays a crucial role in various fields such as software development, artificial intelligence, cybersecurity, data science, and web technologies. Despite its importance, the methods used to teach programming in many educational institutions are still traditional and limited in scope. Most colleges and schools rely on theoretical teaching methods where students are required to write code on paper or work individually on desktop computers in laboratories. This approach often fails to provide practical exposure to real-world collaborative development environments.

In professional software development, programming is rarely an individual activity. Modern software projects are developed by teams of programmers working together in distributed environments. Developers continuously collaborate, review code, debug problems, and share ideas with team members. However, many educational environments do not provide tools that allow students to experience collaborative programming. As a result, students graduate with theoretical knowledge of programming languages but lack practical experience in team-based software development.

Another major challenge is the lack of infrastructure in many educational institutions. Traditional programming labs require powerful computers, installed compilers, integrated development environments (IDEs), and regular maintenance. Setting up and maintaining such infrastructure can be expensive and time-consuming. Institutions with limited financial resources may struggle to provide updated software tools and computing environments. This creates inequality in access to modern programming tools and learning opportunities for students.

Additionally, many existing coding tools are designed for single-user en-

vironments. While modern platforms such as collaborative editors exist, they are often designed primarily for professional developers rather than students or educational institutions. These platforms may require subscriptions, complex configurations, or high-performance systems, making them less accessible to beginner programmers and students from diverse backgrounds.

Another important issue in programming education is the lack of real-time guidance and feedback. When students practice coding exercises, they often face difficulties in identifying errors or understanding programming concepts. In traditional classroom environments, teachers may not be able to monitor the progress of every student simultaneously. This delay in feedback can slow down the learning process and reduce student motivation. Beginners often spend a large amount of time debugging simple errors without receiving immediate assistance.

Accessibility is another critical concern. Many coding environments require installation of large software packages or powerful hardware systems. Students who use low-end devices or limited internet connectivity may face difficulties accessing these tools. A browser-based solution that works efficiently on basic hardware can significantly improve accessibility and ensure that students from different socio-economic backgrounds can participate equally in programming education.

Therefore, there is a strong need for a lightweight, browser-based real-time collaborative code editor that supports multiple users working together simultaneously. Such a platform should allow students to write, edit, and execute code in real time while collaborating with peers. It should also provide features such as real-time synchronization, AI-assisted code suggestions, voice communication, and secure session management. These features would create an interactive learning environment that encourages teamwork, enhances programming skills, and prepares students for real-world software development practices.

The proposed Real-Time Collaborative Code Editor aims to address these challenges by providing an accessible, collaborative, and intelligent coding platform designed specifically for educational environments. By combining real-time collaboration technologies, artificial intelligence assistance, and cloud-based architecture, the system can bridge the gap between traditional programming education and modern industry practices.

1.2 Scope of Project

The scope of the Real-Time Collaborative Code Editor project focuses on developing a modern coding environment that improves the way programming is taught and practiced. The proposed system aims to create a platform where students, teachers, and developers can work together in a shared coding environment that promotes collaboration, accessibility, and interactive learning. The system is designed to address the limitations of traditional coding environments and provide a scalable solution suitable for educational institutions and collaborative development scenarios.

1. **Accessible Coding Platform** : The project will provide a browser-based and lightweight code editor that does not require installation on the user's device. Users can access the editor directly through a web browser, which reduces the need for complex setup procedures. The platform will operate efficiently even on low-cost machines and basic computing devices. This ensures that students from institutions with limited resources can also practice coding without facing technical barriers.
2. **Real-Time Collaboration** : The platform will support real-time collaboration where multiple users can write, edit, and debug code simultaneously. Changes made by one user will be instantly visible to all other participants in the session. This collaborative environment allows students to work together on programming tasks, share ideas, and solve problems collectively. It also reflects modern software development practices where teamwork is essential.
3. **Multi-User Coding Sessions** : The system will allow users to create or join collaborative coding sessions. Each session will act as a shared workspace where multiple participants can interact with the same codebase. This feature is particularly useful for group projects, coding practice sessions, and collaborative assignments in educational environments.
4. **Teacher Integration** : The system will include features that allow teachers to monitor student activities during coding sessions. Teachers can observe the code written by students in real time and provide guidance whenever required. This improves classroom interaction and enables instructors to identify mistakes quickly and help students correct them effectively.

5. **AI-Powered Assistance** : The editor will incorporate artificial intelligence capabilities that assist users during coding. AI can provide contextual code suggestions, highlight syntax errors, and offer debugging hints. This feature acts as a virtual assistant that helps students learn programming concepts more efficiently and reduces the time required to identify and fix errors.
6. **Code Execution and Output Display** : The system will provide a mechanism to execute code and display program output directly within the editor environment. This allows users to test their programs instantly and understand the results of their code. Real-time output helps users debug errors more effectively and improves their understanding of program behavior.
7. **Communication Features** : The platform may include communication tools such as chat or voice interaction to facilitate better collaboration among participants. These features allow users to discuss coding problems, explain logic, and coordinate tasks while working on the same project. Communication tools make collaborative coding sessions more interactive and productive.
8. **Cross-Platform Compatibility** : The system will be designed to work across different operating systems and devices, including Windows, Linux, and macOS. Since the editor is browser-based, users can access it from various devices without worrying about compatibility issues. This flexibility ensures that users can participate in coding sessions from any location.
9. **Secure and Controlled Access** : Security and access control will be implemented to ensure that coding sessions remain protected. Only authorized users will be able to join collaborative sessions. Authentication mechanisms and session management features will help maintain the privacy and security of user data and projects.
10. **Future Expansion and Scalability** : The platform is designed with future expansion in mind. Additional features such as coding competitions, hackathons, project version control, and performance analytics can be integrated in future versions of the system. This scalability ensures that the platform can evolve into a more comprehensive collaborative coding ecosystem.

1.3 Detail of Problem Domain

The Real-Time Collaborative Code Editor project lies at the intersection of several technological and educational domains. The goal of the project is to address challenges faced in programming education and collaborative software development by using modern web technologies and intelligent systems. The following domains describe the problem space in which the project operates.

1. **Educational Technology (EdTech)** - The project falls under the domain of Educational Technology (EdTech), where digital tools are used to improve learning experiences. EdTech aims to make education more interactive, accessible, and efficient by integrating technology into teaching methods. In programming education, however, many institutions still rely on outdated systems that do not fully utilize modern collaborative tools. A real-time collaborative coding platform can enhance engagement and allow students to learn programming concepts through practical interaction rather than theoretical learning alone.
2. **Computer Science Education** - Computer science education forms another important part of the problem domain. Many programming courses focus mainly on syntax, theory, and individual assignments. Students are often evaluated through written examinations or isolated coding exercises. As a result, they may not gain practical exposure to real-world development environments where teamwork, debugging, and collaboration are essential. Bridging this gap between academic learning and professional software development is a key challenge addressed by this project.
3. **Collaborative Software Development** - In modern software engineering, most projects are developed by teams rather than individual programmers. Developers frequently collaborate by sharing code, reviewing contributions, and solving problems together. However, students rarely experience collaborative coding environments during their academic training. This project aims to introduce students to collaborative development practices by allowing multiple users to work on the same code simultaneously in real time.
4. **AI-Assisted Learning** - Artificial Intelligence is increasingly being used to support learning by providing personalized assistance and au-

tomated feedback. In programming education, AI can help detect errors, suggest improvements, and explain coding concepts. Despite these benefits, many educational coding tools still lack integrated AI support. By incorporating AI-powered suggestions and debugging assistance into the collaborative editor, the system can provide intelligent guidance to users during coding sessions.

5. **Accessibility and Inclusivity in Learning** - Accessibility is an important aspect of modern educational systems. Many existing programming tools require expensive hardware or paid subscriptions, which can limit access for students from economically weaker backgrounds. The proposed system focuses on developing a lightweight and browser-based solution that can run on basic devices. This ensures that all students, regardless of their financial background, can access the platform and practice programming.
6. **Cloud-Based Computing** - The project also relates to the domain of cloud computing, where applications and services are delivered over the internet. Cloud-based platforms allow users to access tools and resources without installing software locally. By using cloud technologies, the collaborative code editor can provide real-time synchronization and shared workspaces for multiple users. This reduces the dependency on local hardware and improves system scalability.
7. **Web Application Development** - Web technologies play a crucial role in building modern collaborative platforms. Technologies such as JavaScript frameworks, real-time communication protocols, and web-based editors enable interactive coding environments that run directly in a browser. The problem domain includes designing efficient web applications that support real-time data exchange, user interaction, and scalable collaboration.
8. **Real-Time Communication Systems** - Real-time communication systems enable instant data transfer between multiple users connected to the same application. In collaborative coding environments, every change made by a participant must be synchronized immediately with other users. Technologies such as WebSockets or real-time messaging protocols are used to ensure smooth and efficient communication between clients and servers.
9. **Human-Computer Interaction (HCI)** - Human-Computer Interaction focuses on designing systems that are easy to use and improve

user experience. In coding platforms, the interface should be intuitive, responsive, and user-friendly so that beginners can easily navigate the system. A well-designed interface improves usability and ensures that users can focus on coding rather than struggling with complex tools.

10. **Modern Software Development Practices** - The project also reflects modern software development practices such as collaborative coding, continuous feedback, and shared development environments. These practices are widely used in the software industry and help teams build high-quality applications efficiently. Introducing these practices into educational platforms prepares students for professional development environments and enhances their practical programming skills.
11. **Version Control and Code Management** - Version control is an important aspect of modern software development. Developers often need to track changes in code, maintain different versions of a project, and collaborate with multiple team members. Traditional educational coding environments usually do not provide version management features, which limits students' understanding of real-world development workflows. Integrating version control concepts within a collaborative editor can help users track modifications, revert to previous versions, and maintain organized project structures during collaborative coding sessions.
12. **Remote Learning and Online Collaboration** - With the increasing adoption of remote learning and online education, collaborative digital platforms have become essential for effective teaching and learning. Many educational institutions now conduct programming classes, coding workshops, and hackathons in virtual environments. However, existing tools may not provide seamless collaboration for coding activities. A real-time collaborative code editor can support remote learning by allowing students and instructors to interact, share code, and solve problems together even when they are physically located in different places.

1.4 Gantt Chart

A Gantt chart is a visual project management tool that displays tasks, their duration, and dependencies on a timeline. It helps in tracking progress, scheduling tasks, and improving team coordination.

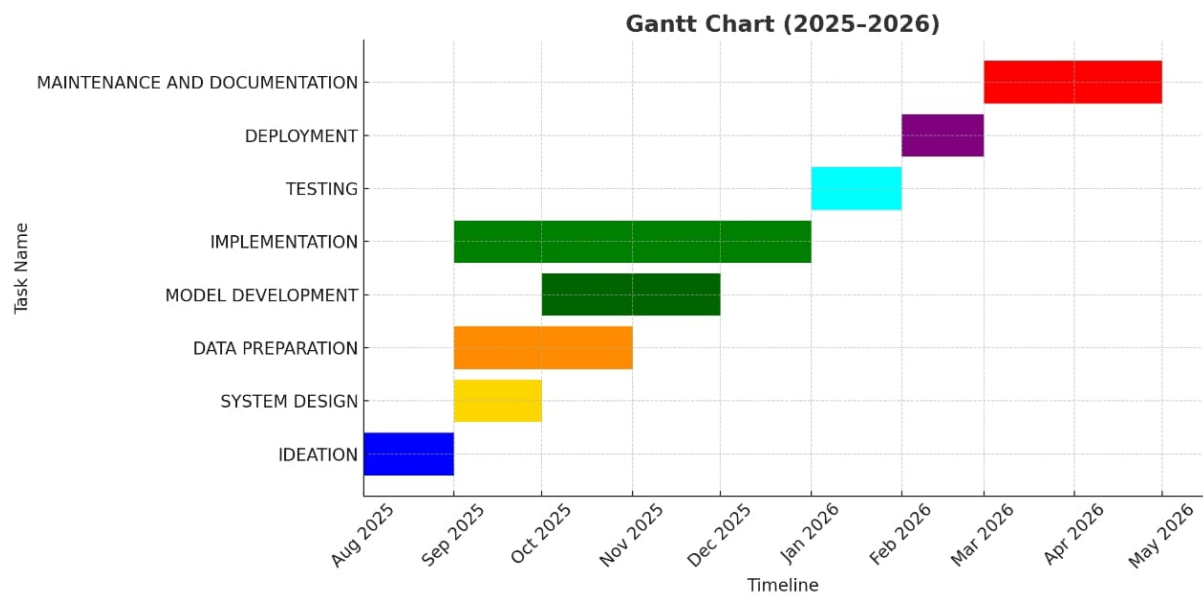


Figure 1.1: Gantt Chart

1.5 System Requirements

Hardware Requirements :

1. **Processor:** Minimum Intel i3 or equivalent, Recommended Intel i5/i7 for smooth multitasking.
2. **RAM:** Minimum 4 GB, Recommended 8 GB or above for handling real-time collaboration.
3. **Storage:** At least 250 GB HDD/SSD, with 1 GB free space for project-related files.
4. **Display:** Minimum resolution 1366×768 , Recommended Full HD for better UI experience.
5. **Internet:** Stable broadband connection (minimum 2 Mbps, recommended 10 Mbps).

Software Requirements :

1. **Operating System:** Windows 10 or higher / Linux / macOS (cross-platform).
2. **Programming Languages:** JavaScript (for frontend & backend), Node.js runtime.
3. **Frameworks/Libraries:** React.js for frontend, Express.js for backend, Socket.IO or WebRTC for real-time communication.
4. **Database:** MongoDB or Firebase (for storing user data and projects).
5. **Editor/IDE Support:** Browser-based Monaco Editor (used in VS Code).
6. **AI Support:** Integration with OpenAI/LLM APIs for chatbot assistance.
7. **Others:** Web browser (Google Chrome/Firefox/Edge).

1.6 Project Report Outline

Chapter 2 – Literature Review

Discusses related studies, existing solutions, and research gaps. Highlights the need for the proposed system and justifies its development.

Chapter 3 – Methodology Used

Explains the approach, system architecture, diagrams (class, use case, ERD), and tools used. Describes how the project requirements are translated into a working solution.

Chapter 4 – Designing of Project

Focuses on detailed design aspects including UI/UX, flowcharts, and screen designs. Demonstrates how each module and component is structured and implemented.

Chapter 5 – Detailed Designing of Project

Provides an in-depth explanation of functionalities, coding logic, and workflows. Includes detailed explanations of real-time collaboration, AI integration, and other key features.

Chapter 6 – Results and Discussion

Presents screenshots, outputs, and discussions of the implemented system. Analyzes how the objectives have been achieved and evaluates performance.

Chapter 7 – Further Work

Discusses possible enhancements, future extensions, and improvements for the system.

Appendices

Includes supplementary materials such as code snippets, diagrams, and additional references.

Chapter 2

Literature Review

2.1 Related Study

Aditya Kurniawan et al. [1]: This paper introduced CodeR, a web-based real-time code editor application developed to support collaborative programming among developers. The system uses WebSocket technology to enable continuous communication between users so that multiple participants can edit and view code simultaneously. CodeR allows users not only to write code but also to execute programs and view the output through an integrated terminal within the same shared workspace. The platform also includes features such as real-time chat communication and terminal building, which allow users to discuss problems and run commands during collaboration. The application supports programming languages such as C, C++, and Java, making it suitable for various development and learning scenarios. The authors emphasize that web-based collaborative coding platforms play a crucial role in modern software development because they allow developers to work together efficiently from different locations without requiring complex installations.

Shivam Chowdhury et al. [2]: The research focused on the design and development of a real-time collaborative code editor specifically intended for technical interview environments. Their research highlights how real-time coding platforms can improve the effectiveness of remote technical assessments. By using WebSocket technology, the proposed application enables multiple users to collaborate instantly on coding tasks within a shared environment. The platform provides a workspace where users can create, edit, compile, and run source code during the interview process. Additionally, the application includes real-time chat functionality and terminal support, allowing interviewers and candidates to communicate easily and evaluate coding performance effectively. The authors suggest that

such collaborative platforms provide a more interactive and efficient environment for conducting technical interviews, particularly in remote hiring scenarios.

Khushwant Viridi et al. [3]: This paper examined the development and challenges associated with collaborative code editors, especially those designed for real-time multi-user coding and knowledge sharing. The study provides an overview of the technologies and protocols used in collaborative platforms, including WebSocket communication and Operational Transformation (OT) algorithms, which help synchronize code changes among multiple users. The authors identify several challenges that arise in real-time collaborative environments, such as latency issues, synchronization conflicts, and the complexity of resolving overlapping edits. To address these challenges, the study presents a prototype collaborative code editor and evaluates its usability and performance compared with existing platforms. The research also highlights the need for improved synchronization mechanisms and more efficient communication protocols to enhance collaborative coding systems.

Mohd Saad Ansari [4]: The study proposed a minimalistic browser-based real-time collaborative code editor developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) along with Socket.IO for real-time communication. The platform allows multiple users to write, edit, view, and execute code simultaneously within a shared environment. This collaborative system is designed to support scenarios such as peer programming, coding interviews, and programming education. The research emphasizes the importance of simplicity, responsiveness, and performance when designing collaborative programming tools. By focusing on a lightweight architecture and intuitive interface, the system aims to provide a smooth and efficient coding experience for users participating in real-time collaboration sessions.

Anup Lal Yadav et al. [5]: This research explored the design and development of an advanced collaborative code editor prototype that integrates WebSocket communication with Operational Transformation algorithms. The study begins with a detailed review of previous research on collaborative coding environments in order to understand the evolution of such systems and the technologies that support them. The authors highlight important technical challenges such as latency, synchronization delays, and

conflict resolution when multiple users edit the same document simultaneously. Their proposed system addresses these issues by improving synchronization mechanisms and ensuring consistent document states across all users. The research demonstrates how advanced collaborative tools can improve knowledge sharing and productivity among distributed software development teams.

Krishna Aher Prof. Jagruti R. Mahajan [6]: This paper presented the development of a real-time collaborative code editor web application using technologies such as React.js, Node.js, and Socket.IO. The application enables developers to work together on the same codebase in real time regardless of their physical location. The system includes features such as syntax highlighting, automatic code formatting, code versioning, and real-time error highlighting to improve coding efficiency and readability. These features help developers quickly identify errors and maintain well-structured code during collaboration. The authors also discuss future enhancements, including the integration of artificial intelligence for intelligent code suggestions and auto-completion, machine learning algorithms for automatic error detection and correction, and immersive technologies such as virtual reality (VR) or augmented reality (AR) to create advanced collaborative programming environments.

Anup Lal Yadav et al. [7]: This research paper conducted another study focusing on the technological foundations and advancements in collaborative code editing systems. The research investigates the key technologies that enable real-time collaborative programming and examines the challenges faced in such environments. Technologies such as WebSocket communication and Operational Transformation algorithms are discussed as essential components for synchronizing edits across multiple users. The authors emphasize the need for continuous innovation in collaborative programming platforms to address ongoing issues such as synchronization complexity, network latency, and conflict resolution. Their study also suggests improvements that could enhance collaboration and knowledge sharing among distributed software development teams.

R. Pathak et al. [8]: The authors analyzed the trends and future prospects of collaborative coding applications and introduced V-Code, an advanced platform designed for remote programming collaboration. The application is developed using modern web technologies such as Nuxt.js

and Python Flask, which enable efficient communication between the client and server. V-Code integrates machine learning models from OpenAI to provide intelligent coding assistance, including code suggestions and auto-completion features. These AI capabilities improve the overall programming experience by helping developers write code faster and reduce errors. The research highlights the increasing demand for intelligent real-time coding platforms in modern software development environments.

Sundarajan Narayanan et al. [9]: This study examines the importance of collaborative coding environments in modern software development and programming education. The authors explain how cloud-based code editors allow developers to collaborate in real time without installing complex development environments on their local systems. Such platforms enable teams to work together from different geographical locations, share knowledge, and debug programs more efficiently. The study concludes that real-time collaborative tools significantly improve productivity and communication among developers working in distributed teams.

Vaithilingam et al. [10]: The research investigated the impact of AI-assisted programming tools on developer productivity. Their research explores how intelligent code assistants can provide suggestions, identify potential errors, and recommend improvements while programmers are writing code. These AI tools analyze the structure and context of the code to generate relevant recommendations that help developers write better programs. The authors conclude that integrating artificial intelligence into development environments can significantly improve coding efficiency and support beginners in learning programming concepts more effectively.

Balik et al. [11]: This paper conducted a comparative study of several online Integrated Development Environments (IDEs) used in programming education. The research evaluates different web-based coding platforms based on criteria such as usability, collaboration capabilities, system performance, and accessibility. The authors found that web-based development environments provide a flexible learning platform for students because they can be accessed from any device with an internet connection. The study also highlights the importance of collaborative features in programming education, as they enable interactive learning and teamwork among students.

Fagerlund et al. [12]: This research focused on the role of artificial intelligence in programming education. The authors examined various AI-powered educational tools designed to assist students in learning programming concepts more effectively. These tools can analyze code written by students, identify errors, and suggest possible solutions or improvements. By providing real-time guidance, AI systems can help students understand programming logic and improve their problem-solving skills. The study concludes that AI-based educational tools have significant potential to enhance learning outcomes in computer science education.

Grover and Pea [13]: Grover and Pea discussed the concept of computational thinking and its importance in modern education systems. The authors explain that programming education helps students develop analytical reasoning, logical thinking, and problem-solving abilities. The study emphasizes that students learn programming concepts more effectively when they are provided with interactive environments that encourage experimentation and collaboration. Such environments allow learners to test ideas, make mistakes, and learn through problem-solving activities.

Bennedsen et al. [14]: This research investigated the role of collaborative learning in programming education. Their research analyzed how group-based programming activities influence student engagement and understanding of programming concepts. The findings show that collaborative coding environments encourage communication among students and promote peer-to-peer learning. By working together on coding tasks, students are able to share ideas, solve problems collectively, and gain a deeper understanding of programming concepts.

Sentance and Waite [15]: The authors explored different teaching approaches used in programming education at schools and universities. The study identifies several difficulties commonly faced by students, such as understanding programming syntax, debugging errors, and learning complex programming concepts. The authors suggest that interactive coding platforms and collaborative learning environments can help overcome these challenges. By providing immediate feedback and opportunities for collaboration, such tools make programming education more engaging and effective.

Guzdial [16]: This study focused on the role of computing education in

preparing students for careers in the technology industry. The author highlights the importance of providing practical programming environments that allow students to experiment, collaborate, and develop real-world software projects. The study suggests that integrating modern coding tools and collaborative platforms into educational systems can significantly improve the quality of programming education and better prepare students for future technological challenges.

2.2 Research Gaps

Despite several studies and developments in collaborative coding platforms and intelligent code editors, there remain critical gaps that justify the need for further research. Many existing systems focus on professional development environments but fail to address the specific requirements of educational institutions and beginner programmers. The following research gaps highlight the limitations of current solutions and the need for improved collaborative coding platforms.

1. **Limited Real-Time Collaboration for Students :** Existing platforms primarily focus on professional developers or general-purpose collaborative editors. There is limited research on developing collaborative coding environments specifically tailored for school or college students. Many tools are designed for experienced programmers and may not consider the learning pace and understanding level of beginners. As a result, students often find it difficult to adapt to such platforms during the learning process.
2. **Integration of AI Assistance :** Although AI-based coding tools such as GitHub Copilot provide automated code suggestions, very few collaborative platforms integrate AI-assisted learning features directly into real-time coding environments. AI assistance can play an important role in guiding beginners by detecting syntax errors, providing debugging suggestions, and explaining programming concepts. The integration of AI within collaborative learning platforms remains an area that requires further research and development.
3. **Cross-Platform Accessibility :** Most existing solutions are designed either as web-based applications or desktop-based development environments. This limits accessibility for users who want to work across different devices or operating systems. A major research gap exists in developing seamless cross-platform collaborative editors that allow users to access the same coding environment from multiple devices without synchronization issues or performance delays.
4. **Scalability and Performance Evaluation :** Many research studies focus on the development of collaborative coding tools but do not perform detailed evaluations of system scalability. Real-time synchronization becomes increasingly complex when a large number of users participate in the same coding session. Handling multiple concurrent

users while maintaining low latency and data consistency remains a major technical challenge that needs further investigation.

5. **User Engagement and Interaction Analysis :** While collaborative coding platforms enable multiple users to work together, limited research has been conducted on how these platforms influence student engagement and interaction. Understanding how collaborative tools affect learning outcomes, participation levels, and peer-to-peer learning is essential for improving educational coding platforms. More studies are needed to analyze both qualitative and quantitative impacts of collaborative coding environments.
6. **Security and Privacy in Educational Contexts :** Security and privacy are important considerations when designing collaborative platforms for students. Existing research often focuses on technical functionality but gives limited attention to issues such as data protection, user authentication, role-based access control, and secure communication between participants. Addressing these security challenges is necessary to ensure a safe and reliable collaborative learning environment.
7. **Customization for Learning Objectives :** Most collaborative editors are designed for general-purpose programming and do not provide customization features aligned with educational objectives. For example, beginner students may require simplified interfaces, programming hints, and interactive debugging assistance. There is a lack of platforms that adapt their features according to different levels of programming knowledge and learning goals.
8. **Limited Support for Interactive Learning :** Many coding platforms allow users to write and execute code but do not include interactive learning features such as guided exercises, automated feedback, or step-by-step explanations. Interactive learning tools can help students understand programming concepts more effectively. The absence of such features creates a gap between coding practice and conceptual understanding.
9. **Lack of Integrated Communication Tools :** Collaboration in programming often requires communication between participants. However, many existing collaborative coding tools lack integrated communication features such as chat or voice interaction. Without effective

communication tools, collaboration becomes less efficient, especially in remote learning environments where participants cannot interact face-to-face.

10. **Limited Focus on Educational Analytics :** Another research gap is the lack of analytics and performance tracking features in collaborative coding environments. Educational analytics can help teachers monitor student progress, identify learning difficulties, and evaluate participation levels during coding sessions. Incorporating analytics into collaborative coding platforms can provide valuable insights for improving both teaching strategies and learning outcomes.

2.3 Objective of Project

The primary objective of this project is to design and develop a Real-Time Collaborative Code Editor that enhances programming education and collaborative software development. The system aims to provide an interactive environment where multiple users can write, edit, and execute code simultaneously while receiving intelligent assistance and guidance. The following objectives describe the major goals of the proposed system.

1. **Develop a Real-Time Collaborative Environment :** The first objective is to design a web-based platform that allows multiple users to write, edit, and execute code simultaneously. The system should support real-time synchronization so that changes made by one user are instantly reflected in the editors of other participants. This feature promotes teamwork and collaborative learning among students.
2. **Integrate AI-Assisted Learning Features :** Another important objective is to integrate artificial intelligence capabilities into the coding platform. AI assistance can provide code suggestions, syntax corrections, and debugging hints. These features help beginners understand programming concepts more easily and improve their coding efficiency.
3. **Ensure Cross-Platform Accessibility :** The system aims to provide a platform that can be accessed from different devices and operating systems. Since the editor is browser-based, users can access it through desktops, laptops, or tablets without installing additional software. This improves accessibility and ensures that students can participate in coding sessions from various environments.
4. **Improve Programming Learning Experience :** The platform is designed to enhance the overall learning experience of students studying programming. By providing interactive coding environments, real-time feedback, and collaborative features, the system helps students develop practical programming skills rather than relying solely on theoretical learning.
5. **Enable Real-Time Communication Between Users :** An objective of the project is to enable communication among participants during collaborative coding sessions. Features such as chat or voice communication allow users to discuss problems, share ideas, and coordinate tasks while working on the same codebase.

6. **Maintain Security and Data Privacy :** Security is an essential objective of the proposed system. The platform should ensure that only authorized users can access collaborative coding sessions. Proper authentication mechanisms and secure data handling techniques will be implemented to protect user information and coding projects.
7. **Support Multiple Programming Languages :** The system aims to support multiple programming languages such as Python, Java, C++, and JavaScript. This flexibility allows users to practice coding in different languages and makes the platform useful for a wide range of programming courses and learning activities.
8. **Provide Real-Time Code Execution and Outputs :** Another objective is to enable users to execute their code directly within the editor and view program outputs instantly. This feature helps users test their programs, identify errors quickly, and understand the behavior of their code.
9. **Enhance Collaboration for Educational Institutions :** The platform is designed to support collaborative learning activities in educational institutions. Teachers can organize coding sessions, monitor student progress, and provide real-time guidance during programming exercises. This improves classroom interaction and supports effective teaching methods.
10. **Provide a Scalable and Extensible System :** The final objective is to design the system in a scalable manner so that it can support a growing number of users and features in the future. The platform can later be extended to support coding competitions, project management tools, and integration with external development platforms.

Chapter 3

Methodology Used

Public Datasets : One should begin to look for public data-sets that have something to do with brain MRI scans.

This section describes the methodology proposed to develop a Realtime Collaborative Code Editor. The study aims to provide students with an interactive environment where multiple users can code simultaneously, receive AI-assisted guidance, and collaborate in real time. The system is designed to improve coding learning, foster teamwork, and provide immediate feedback on code execution.

The designed system takes input from users through a web interface and performs multiple stages, including real-time synchronization, AI assistance, syntax handling, and code execution. Each process is described in detail below.

Flow Diagram: Realtime Collaborative Code Editor

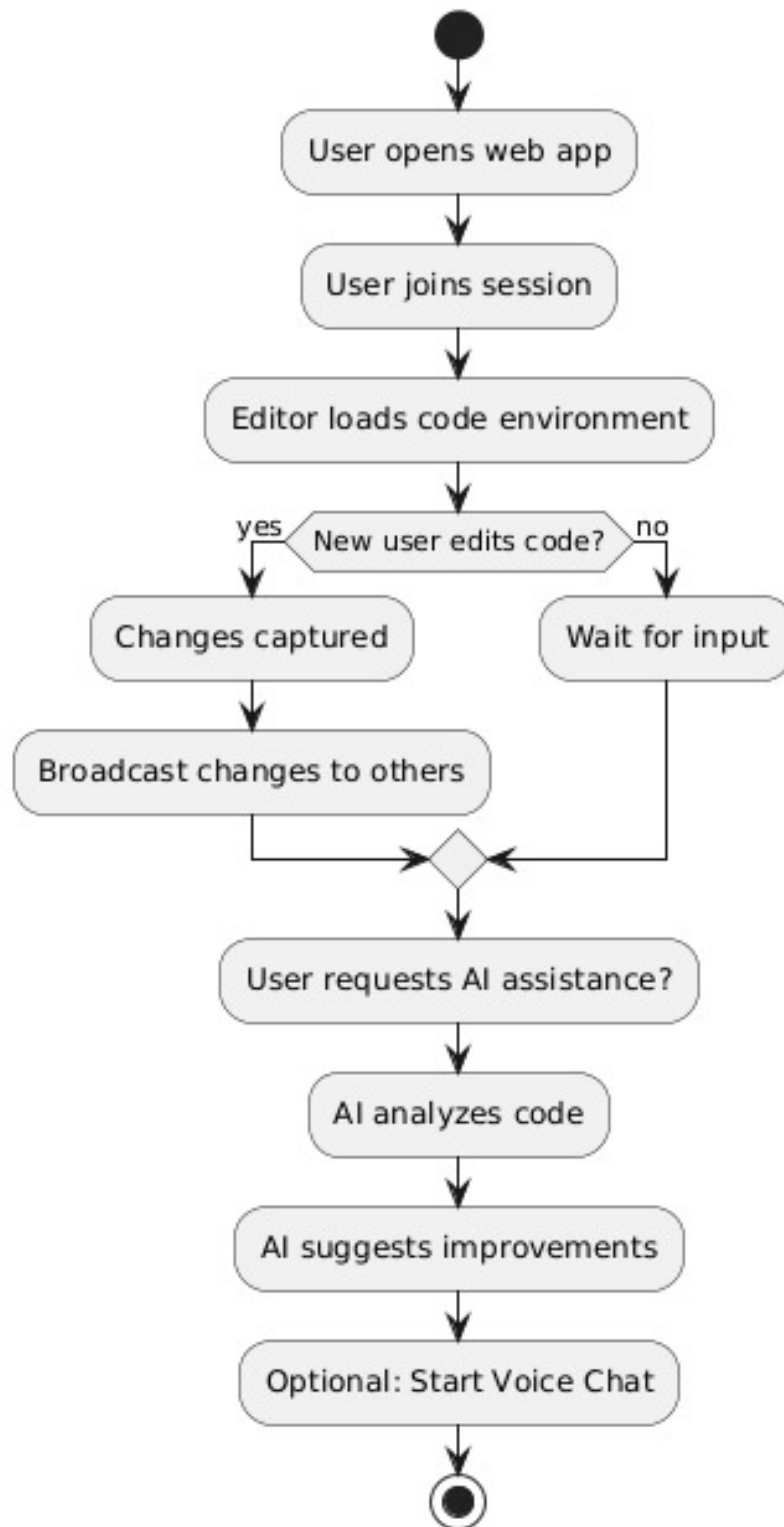


Figure 3.1: Flow Diagram of Proposed Realtime Collaborative Code Editor

3.1 Code Input and User Interaction

Code input is the initial step in the proposed system. The editor is accessed by users through a web-based interface where they can begin writing code using their preferred programming languages such as Python, Java, C++, or JavaScript. Since the platform operates in a browser environment, users do not need to install any additional software or development tools. This improves accessibility and allows students to start coding immediately.

The code editor continuously records user inputs such as keystrokes, cursor movements, and code modifications. These inputs are transmitted to the server through real-time communication channels. The server processes these updates and synchronizes them across all connected participants in the collaborative session. This ensures that all users see the same code in real time.

To enhance usability, the editor includes several features commonly found in modern integrated development environments. These features include line numbering, syntax highlighting, auto-completion suggestions, and customizable editor themes. Such features improve readability and help users write code more efficiently.

The system also provides session-based collaboration, where multiple participants can join the same coding workspace. Each user can observe the changes made by others instantly. This collaborative environment encourages teamwork and knowledge sharing among participants.

Example: When three students are working on a Python script collaboratively, any alteration created by one of them is immediately updated in the editor windows of the other two students. This allows them to work together on the same code without creating multiple conflicting versions.

3.2 Real-Time Synchronization

Real-time synchronization is essential for maintaining consistency among multiple users working within the same collaborative coding session. Without synchronization mechanisms, users editing the same file simultaneously could overwrite each other's changes, leading to inconsistencies and possible data loss. The proposed system addresses this problem using Conflict-Free Replicated Data Types (CRDTs) implemented through the Yjs library. CRDT is a distributed data structure

that allows multiple users to modify shared data simultaneously without creating conflicts. The system integrates CRDT with real-time communication protocols such as WebSocket or Socket.IO in order to maintain low-latency communication between the client and the server. The synchronization mechanism works through several components. The CRDT algorithm ensures that concurrent edits from different users are automatically merged without conflicts, allowing users to edit the same document at the same time. WebSocket communication provides a persistent two-way communication channel between the client and the server, which allows updates to be transmitted instantly between users. In addition, session management maintains synchronization among all users connected to the same collaboration session and ensures that everyone is viewing the same version of the code. This approach eliminates the need for traditional locking mechanisms where only one user can edit a file at a time. Instead, multiple users can modify different sections of the code simultaneously, and the system automatically merges the changes. For example, if one student inserts a new function while another edits a variable in the same program, the CRDT algorithm combines both modifications and updates the shared document without losing any changes.

3.3 Pre-processing and Syntax Handling

Before code execution or AI-assisted analysis, the input code goes through a preprocessing stage. This preprocessing ensures that the code is properly structured and formatted before being processed further. The preprocessing stage plays an important role in improving code readability and reducing the chances of runtime errors. Several operations are performed during preprocessing. Syntax highlighting is applied so that programming keywords, variables, and functions are displayed using different colors, which helps users easily understand the structure of the code. The system also performs real-time error detection by identifying syntax mistakes and highlighting problematic lines of code immediately. In addition, automatic code formatting is applied to maintain proper indentation and spacing throughout the program. Auto-indentation ensures that nested structures such as loops and conditional statements follow correct indentation rules. These preprocessing features help users quickly identify mistakes and understand the structure of their programs. They also improve the

effectiveness of AI suggestions because properly formatted code is easier for the system to analyze. For instance, in Python programming, indentation errors are very common. If a user forgets to indent a statement inside a loop, the editor immediately highlights the error and suggests the correct indentation.

3.4 AI-Assisted Code Suggestions

The proposed system integrates an AI-assisted module that provides intelligent guidance during the coding process. This AI module analyzes the code written by users and provides contextual suggestions that help improve coding efficiency and learning. The AI system supports several functionalities that assist users during programming. It provides code autocompletion, which suggests function names, variables, and commonly used programming constructs while the user is typing. The system also offers contextual hints based on the programming pattern being used, helping users write more efficient code. Additionally, the AI module performs error detection by identifying logical or syntax errors and recommending possible corrections. It can also provide explanations for complex code segments so that beginners can better understand programming concepts. The AI assistant is designed to guide learners rather than solving problems completely for them. Instead of directly giving full solutions, it offers hints and suggestions that encourage users to think and arrive at the correct solution themselves. This approach promotes active learning and strengthens problem-solving skills. For example, while writing a loop structure, the AI module can suggest the correct syntax and possible conditions based on the code that has already been written.

3.5 Feature Extraction and Analysis

Feature extraction involves identifying important components of the code that can be analyzed to enhance the collaborative coding experience. The system examines the structure of the program and extracts key elements that help the AI module understand the programming logic and provide better suggestions. The extracted features include functions and methods defined in the program along with their parameters. The system also detects loops and control structures such as conditional statements and iteration blocks. In addition, it analyzes variable usage, including how variables are declared and used within

different scopes of the program. Another important feature extracted is code patterns, which involves recognizing commonly used programming constructs that appear frequently in the code. These extracted features help the AI module generate more accurate suggestions and provide meaningful feedback to users. They also allow the system to track coding patterns and identify areas where users may require additional guidance. For example, if a student repeatedly misuses loop conditions, the system can detect this pattern and provide targeted suggestions to help improve the student's understanding.

3.6 Comparison and Conflict Resolution

When multiple users edit the same code file simultaneously, conflicts may arise due to overlapping changes. The proposed system handles these conflicts using the CRDT synchronization mechanism. This mechanism ensures that all edits are properly integrated without causing inconsistencies. The conflict resolution process ensures that no edits are lost, all participants see the same version of the code, and collaboration continues smoothly without interruptions. Unlike traditional systems that rely on file locking mechanisms, the CRDT-based approach allows multiple users to edit the document simultaneously. The system automatically integrates the changes made by different users without overwriting previous edits. For example, if two users edit the same line of code at the same time, the CRDT algorithm merges both changes in a consistent order and updates the shared document so that all collaborators can see the final integrated version.

3.7 Code Execution and Output Display

After the code has been written and processed, the system allows users to execute their programs in a secure environment. The code execution module runs the program inside a sandbox environment, which prevents malicious code from affecting the system or other users. This execution module provides several features that help users understand the results of their programs. A console output panel displays the output generated by the program during execution. The system also highlights runtime errors and provides relevant messages that help users identify and correct issues. Additionally, the platform supports multiple programming languages, allowing users to execute different

types of programs within the same environment. Execution logs are also maintained to record program execution details, which can assist in debugging and analysis. All users participating in the collaboration session can view the execution results in real time. This allows collaborators to understand the program output together and work collectively to fix errors. For example, if a Python script prints numbers from 1 to 10, all collaborators will immediately see the output displayed in the console panel.

3.8 Dataset

To evaluate the performance and functionality of the collaborative coding platform, a collection of programming exercises and sample datasets is used. These datasets simulate different levels of programming complexity as well as collaborative coding scenarios. The dataset includes beginner-level programs that focus on simple exercises involving loops and conditional statements. It also contains intermediate-level tasks that require collaborative problem-solving and involve functions and algorithms. In addition, advanced programs are included to test complex scenarios and evaluate the effectiveness of AI-assisted suggestions. These datasets help measure important factors such as system latency, synchronization accuracy, AI performance, and user engagement within collaborative coding sessions.

3.9 Preprocessing of Code

Preprocessing ensures that the code entered by users is properly structured before execution or further analysis. This stage prepares the code by validating its syntax and formatting so that it can be processed more effectively by the system. Several tasks are performed during this stage. The system validates the code to detect syntax errors before execution begins. It also normalizes the code by standardizing formatting and structure. Error handling mechanisms identify potential runtime issues that may occur during execution. Additionally, the code is tokenized, which means it is broken down into smaller meaningful components that can be analyzed by the system. This preprocessing stage improves overall system reliability and ensures that the code is ready for further processing.

3.10 Testing and Evaluation Sets

To evaluate the performance of the system, three different sets of programs are used: the training set, validation set, and testing set. Each set plays an important role in assessing the effectiveness of the collaborative coding platform. The training set contains programs that are used to train the AI module so that it can learn programming patterns and generate useful suggestions. The validation set is used to adjust and fine-tune the AI suggestions as well as real-time collaboration features. The testing set consists of independent programs that are used to evaluate system performance, accuracy of error detection, and overall user experience. These evaluation sets allow developers to measure the efficiency of the system and identify areas where improvements may be needed.

3.11 AI Module Implementation

The AI module is implemented using pre-trained machine learning models or APIs such as OpenAI Codex. This module processes the code written by users and generates intelligent suggestions based on programming patterns and best practices. The AI module supports several important features including code completion, syntax suggestions, debugging hints, and explanations of complex code segments. These features help users write code more efficiently and understand programming concepts more clearly. By providing intelligent assistance during coding, the AI module helps reduce the time required to debug errors and improves the overall learning experience for users.

3.12 Summary of Methodology

The proposed methodology combines collaborative programming techniques with artificial intelligence to create an efficient and interactive coding platform. The system allows multiple users to work on the same code simultaneously while maintaining synchronization and data consistency. Through the integration of CRDT-based synchronization, AI-assisted suggestions, preprocessing mechanisms, and secure code execution, the platform ensures that collaboration remains smooth and effective. Multiple users can edit the same code in real time without conflicts, AI suggestions help improve learning outcomes, and preprocessing features enhance code quality. In addition, the sandbox-based execution environment ensures safe and immedi-

ate code execution. Overall, this methodology provides a structured framework for building a collaborative coding platform that supports both educational learning and real-world software development practices.

Chapter 4

Designing of Project

4.1 0-Level DFD

Figure 4.1 represents the 0-level Data Flow Diagram (DFD) of the Real-time Collaborative Code Editor with AI Assistance. A 0-level DFD, also known as a context diagram, illustrates the overall flow of information between external entities and the main system. It provides a high-level overview of how users interact with the system and how data is processed and delivered to collaborators.

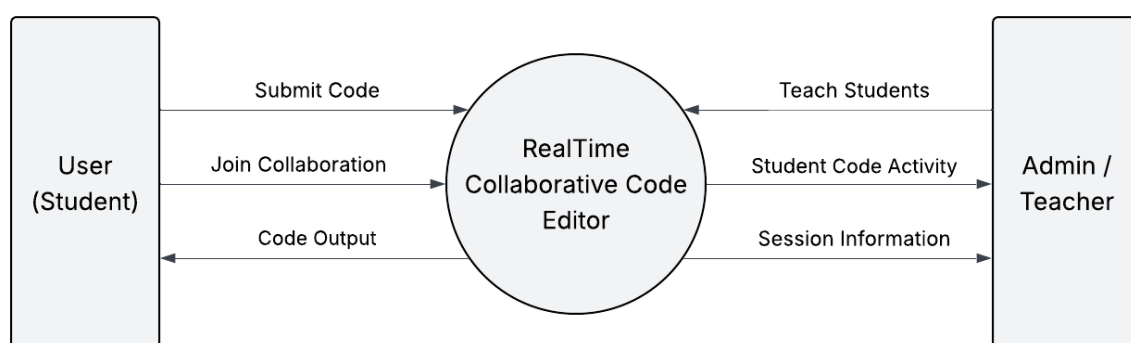


Figure 4.1: 0-level DFD

In this system, the primary entities involved are User (Student), the Real-Time Collaborative Code Editor, and Admin/Teacher. The diagram shows how coding activities, collaboration requests, and administrative monitoring data flow through the system. This high-level representation helps in understanding the basic working of the platform before moving to more detailed system processes.

1. User(Student) :

The process begins with the User, who represents students accessing the collaborative coding platform. Students interact with the system through

a web-based interface where they can write, edit, and collaborate on code in real time.

The user provides inputs such as submitting code and joining collaboration sessions. These inputs are transmitted to the system, where the code is processed and synchronized.

The system then returns outputs such as code execution results to the user.

Example: When a student submits a piece of code or joins a collaboration session, the system processes the request and displays the compiled output or shared code environment to the student.

2. Real-time Collaborative Code Editor System :

The Real-Time Collaborative Code Editor acts as the central component of the system. It receives inputs from users and manages collaborative coding sessions.

The system performs functions such as processing submitted code, enabling collaboration between users, generating code output, and maintaining session information. It also records coding activities so that instructors can monitor student progress.

After processing user inputs, the system produces outputs such as code execution results, student activity information, and session details.

3. Admin / Teacher :

The Admin or Teacher entity represents instructors who manage and monitor coding sessions. Teachers interact with the system by teaching students or supervising coding activities.

The system provides outputs to the teacher such as student code activity and session information, which help instructors monitor progress and guide students during collaborative coding sessions.

For example a teacher can view the coding activity of students during a session and analyze how students are working on coding tasks.

Overall, the 0-level DFD provides a simplified overview of the Real-Time Collaborative Code Editor. It shows how students interact with the system by submitting code and joining collaboration sessions, while teachers monitor student activity and session information. This high-level representation helps in understanding the fundamental interaction between users, the system, and administrators.

4.2 1-Level DFD

Figure 4.2 illustrates the 1-Level Data Flow Diagram (DFD) of the Real-time Collaborative Code Editor with AI Assistance. While the 0-level DFD provides a high-level overview of the entire system, the 1-level DFD breaks the system into smaller functional processes and shows how data flows between them.

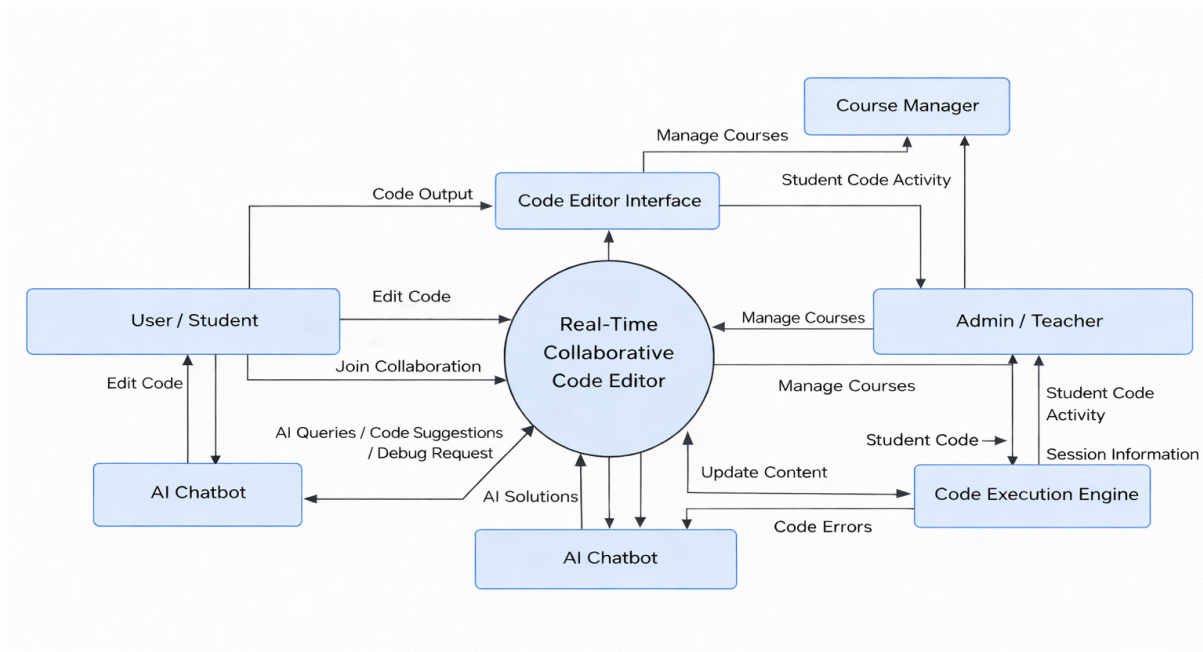


Figure 4.2: 1-level DFD

The diagram shows how students interact with the collaborative coding platform, how code is processed by the editor, and how teachers manage courses and monitor student activities. The system also integrates an AI chatbot for coding assistance and a code execution engine to compile and run the programs.

1. User / Student Interaction :

The process begins with the User or Student, who interacts with the platform through the web-based coding interface. Students can write and edit code directly in the editor and participate in collaborative coding sessions with other users.

The inputs provided by the user include writing or editing code, joining collaboration sessions, and requesting AI assistance for debugging or code suggestions. These inputs are transmitted to the Real-Time Collaborative Code Editor, which processes the requests and manages collaboration

between users.

After processing the requests, the system returns outputs such as code execution results and AI suggestions. These outputs are displayed to the student through the editor interface, allowing users to see the results of their coding activities.

2. Code Editor Interface :

The Code Editor Interface serves as the primary interaction layer between the users and the system. It allows students to write, edit, and view code in a simple and user-friendly environment.

The editor receives updated code from the collaborative system and displays the results back to the users. It also forwards user inputs such as edited code and collaboration requests to the main system for further processing.

Through this interface, students can write and modify source code, view the code output generated after execution, and participate easily in collaborative coding sessions. The processed code is then forwarded to other modules such as the AI chatbot and the code execution engine for further processing.

3. Real-Time Collaborative Code Editor :

The Real-Time Collaborative Code Editor acts as the central process of the system. It manages collaborative coding sessions and coordinates communication between different modules of the platform.

This component receives code updates from students and synchronizes them across the system so that multiple users can work on the same code simultaneously. It ensures that any change made by one user is reflected instantly for all participants in the session.

The collaborative editor also forwards code requests to other subsystems such as the AI chatbot for suggestions and the code execution engine for compiling and running the program. In addition, it manages interaction between users and instructors during coding sessions and ensures that the collaborative environment functions smoothly.

4. AI Chatbot :

The AI Chatbot module provides intelligent assistance to students during coding sessions. It receives inputs such as AI queries, debugging requests, and code suggestions from users through the collaborative editor.

The AI chatbot analyzes the code and provides useful responses such as code suggestions, debugging assistance, and explanations of programming concepts. This helps students understand their mistakes and improve their coding skills.

These AI-generated solutions are returned to the collaborative editor and then displayed to users through the code editor interface, allowing students to quickly resolve errors and continue their coding tasks.

5. Code Execution Engine :

The Code Execution Engine is responsible for compiling and running the code submitted by users. When the collaborative editor sends a code request, the execution engine processes the program and generates the required outputs.

The engine executes the program and produces either successful code output or error messages if the program contains syntax or runtime issues.

These outputs are then sent back to the collaborative editor and displayed to the user through the code editor interface so that students can immediately see the results of their programs.

6. Course Manager and Admin / Teacher :

The Course Manager and Admin/Teacher modules allow instructors to manage courses and monitor student activity within the system.

Teachers can manage courses, update learning materials, monitor student coding activities, and view session information through the platform.

The system collects information related to student code activity and provides this data to instructors. This allows teachers to track student progress and guide them effectively during collaborative coding sessions.

Overall, the Level-1 DFD provides a detailed view of the internal components of the Real-Time Collaborative Code Editor. It explains how user inputs are processed through different modules such as the Code Editor Interface, AI Chatbot, Code Execution Engine, and Course Manager. These modules work together to provide a collaborative and interactive coding environment for students and teachers.

4.3 2-Level DFD

Figure 4.3 illustrates the Level-2 Data Flow Diagram (DFD) of the Real-Time Collaborative Code Editor System. While the Level-1 DFD explains the major modules of the system, the Level-2 DFD provides a more detailed representation of how these modules interact with each other and how data flows between different components of the collaborative coding platform.

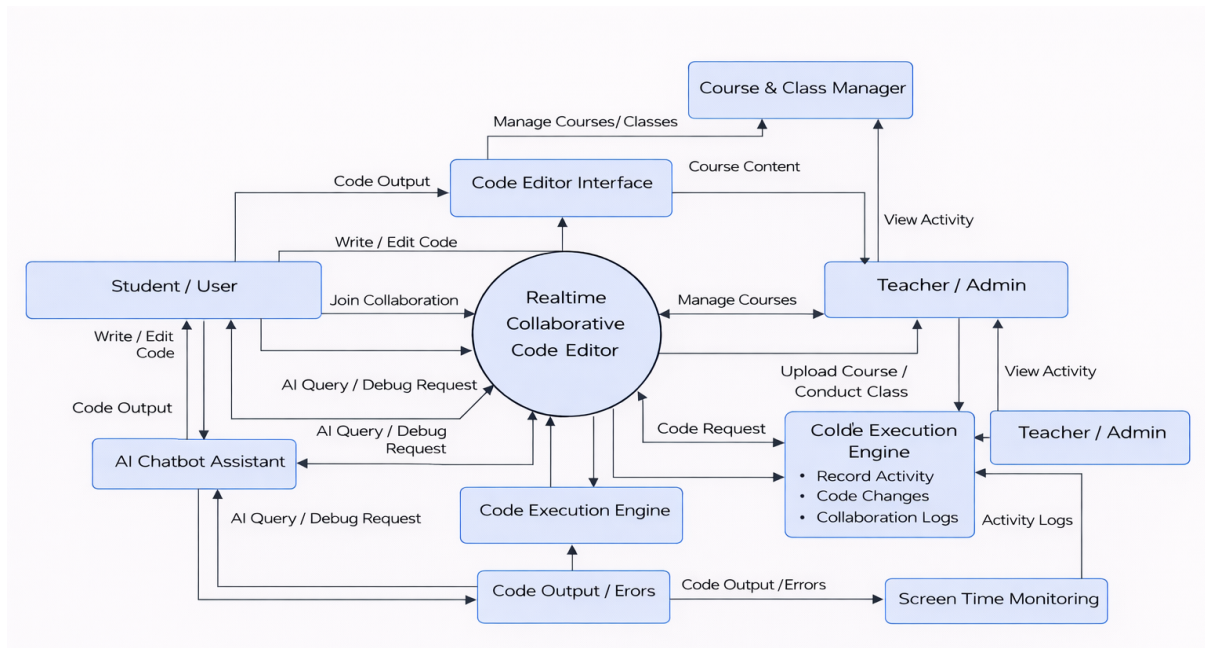


Figure 4.3: 2-level DFD

The Level-2 diagram shows the internal working of the system by expanding the main processes into smaller functional units. It explains how students interact with the collaborative editor, how AI assistance is provided, how code execution takes place, and how teachers manage courses and monitor student activities. The diagram also illustrates the role of the Code Editor Interface, AI Chatbot Assistant, Code Execution Engine, Course and Class Manager, and Screen Time Monitoring module.

1. Student / User Interaction :

The process begins with the Student or User, who accesses the collaborative coding platform through a web-based interface. Students can write or edit code directly in the editor and join collaborative coding sessions where multiple users can work on the same project simultaneously.

When a student writes or modifies code, the request is sent to the Real-

time Collaborative Code Editor, which manages the collaborative session. The editor processes the changes and forwards the updated information to other modules in the system.

Students can also request assistance through the AI chatbot by sending AI queries or debugging requests. These requests are processed by the system and appropriate responses are returned to the user.

2. Code Editor Interface :

The Code Editor Interface acts as the main interaction layer between the user and the collaborative system. It displays the code being written by students and shows the output generated after execution.

When the collaborative editor processes code updates, the results are transmitted to the editor interface, which displays the code output to the student. This interface also allows users to easily edit code and participate in collaborative sessions.

In addition, the editor interface communicates with the Course and Class Manager, which manages course materials and coding sessions organized by instructors.

3. Realtime Collaborative Code Editor :

The Realtime Collaborative Code Editor is the central component of the system and coordinates communication between all modules. It receives user inputs such as code edits, collaboration requests, and AI queries and distributes them to the appropriate subsystems.

This module ensures that all users working in a collaborative session see the same updated version of the code. When one user modifies the code, the system synchronizes the update and shares it with all participants in real time.

The collaborative editor also forwards code execution requests to the Code Execution Engine, sends debugging queries to the AI Chatbot Assistant, and manages communication with the Teacher or Admin for course management and session control.

4. AI Chatbot Assistant :

The AI Chatbot Assistant provides intelligent coding assistance to students during programming sessions. It receives inputs such as AI queries, debugging requests, and requests for code suggestions from users through the collaborative editor.

The chatbot analyzes the code using AI models and generates helpful responses such as debugging recommendations, code improvements, and explanations of programming concepts.

These responses are returned to the collaborative editor and then displayed to the students through the coding interface. This feature helps beginners understand programming concepts and resolve errors more efficiently.

5. Code Execution Engine :

The Code Execution Engine is responsible for compiling and executing the code written by users. When the collaborative editor sends a code request, the execution engine processes the program and runs it in a controlled environment.

After execution, the engine generates results that may include successful code output or error messages if the code contains syntax or runtime errors.

These outputs are then sent back to the collaborative editor and displayed to the user through the editor interface. The execution engine also records coding activities such as code changes and collaboration logs for monitoring purposes.

6. Course and Class Manager :

The Course and Class Manager module allows instructors to organize and manage coding courses within the platform. Teachers can upload course materials, manage classes, and provide coding exercises for students.

This module communicates with the collaborative editor and the editor interface to ensure that students have access to relevant course content. It also helps instructors track the progress of collaborative coding sessions conducted during classes.

7. Teacher / Admin and Activity Monitoring :

The Teacher or Admin interacts with the system to monitor student activities and manage courses. Teachers can view student code activity, track participation in collaborative sessions, and review coding progress.

The system records important information such as code changes, collaboration logs, and session details. These records are processed by the Code Execution Engine and monitoring modules, which generate activity logs for instructors.

Additionally, the system includes a Screen Time Monitoring module

that tracks how long students spend using the coding platform. If the screen time exceeds a defined limit, the system can generate alerts to ensure responsible usage.

4.4 Use Case Diagram

Below Figure 4.4 illustrates the Use Case Diagram of the Real-time Collaborative Code Editor system. A use case diagram represents the interaction between users (actors) and the system functionalities. It helps in understanding how different participants interact with the platform and how the system responds to their actions.

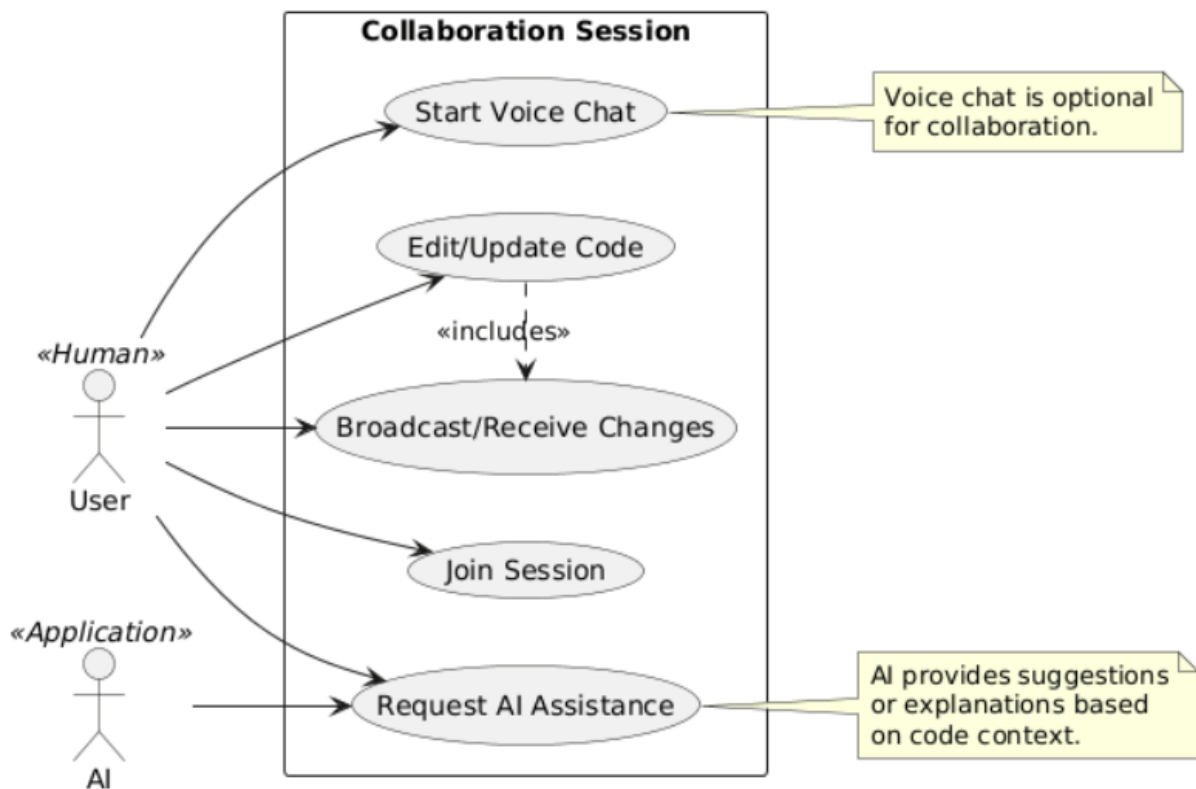


Figure 4.4: Use Case Diagram

In this system, two primary actors are involved: the User and the AI Assistance Module. The diagram highlights the key activities that users can perform within the collaborative coding environment and how the AI module provides support during coding sessions.

1. Role of User :

The User is the main actor who interacts directly with the collaborative code editor. Users may include students, developers, or instructors who participate in coding sessions. Through the platform interface, users can perform several actions that enable collaboration and coding activities within the system.

One of the primary actions performed by the user is joining a session.

A user can join an existing collaboration session or create a new one. A session acts as a shared workspace where multiple participants can work on the same code simultaneously. This allows users to collaborate effectively while solving programming tasks.

After joining a session, the user can write new code or modify existing code in the editor. The system captures these edits and prepares them for synchronization with other users connected to the same session. This ensures that all participants are able to view and work on the latest version of the code.

Whenever a user edits the code, the system automatically broadcasts the changes to all other collaborators connected to the session. At the same time, the user receives updates made by other participants in real time. This real-time synchronization enables smooth collaboration and prevents conflicts between different code edits.

In addition to coding features, users can also initiate voice chat with other participants to improve communication during collaborative coding sessions. Voice communication allows team members to discuss problems, share ideas, and coordinate their work more effectively while developing code together.

Through these interactions, users are able to collaborate efficiently in a shared coding environment and work together on programming tasks.

2. Role of AI Assistance Module :

The AI Assistance Module acts as a supportive component within the system and provides intelligent assistance to users during coding activities. Its primary purpose is to help users improve the quality of their code and resolve programming issues more efficiently.

When users require assistance, they can send a request to the AI module. The system forwards the current code context and relevant information to the AI engine for analysis. Based on this information, the AI module evaluates the code and identifies potential errors or improvements.

The AI module provides suggestions such as correcting syntax errors, recommending better coding practices, or suggesting optimized code structures. These suggestions help users write more efficient and error-free programs.

In addition to debugging assistance, the AI module also supports learning by explaining programming concepts and suggesting alternative approaches to solving coding problems. This feature is particularly useful

for beginner programmers who may need additional guidance while learning new programming concepts.

Overall, the Use Case Diagram demonstrates how users interact with the collaborative coding platform and how AI assistance supports the coding process. It highlights the main functionalities of the system and shows how collaborative coding, AI suggestions, and communication tools work together to create an efficient development environment.

Chapter 5

Detailed Designing of Project

5.1 Class Diagram

The following Figure 5.1 represents the Class Diagram of the Real-time Collaborative Code Editor system. A class diagram is a part of the Unified Modeling Language (UML) used to describe the structure of a system by showing its classes, attributes, methods, and the relationships between them. It provides a clear understanding of how different components of the system interact with each other.

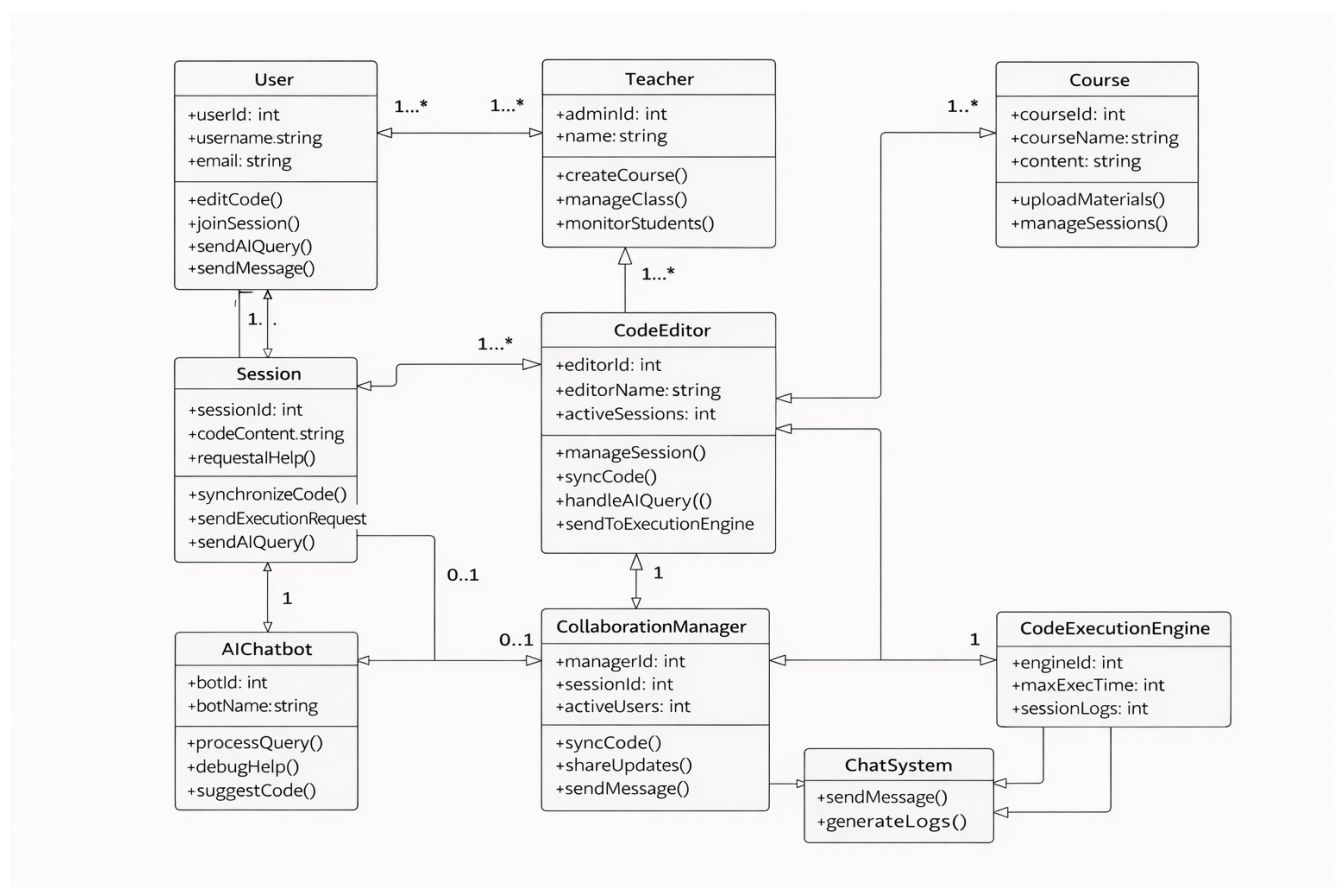


Figure 5.1: Class Diagram

In this project, the class diagram of the collaborative code editor includes major classes such as User, Session, CodeEditor, AIChatbot, Course, and CodeExecutionEngine. These classes represent different components of the system that work together to support collaborative coding, AI assistance, and code execution.

1. User Class :

The User class represents students or individuals who use the collaborative coding platform. Each user has attributes such as `userId`, `username`, and `email`, which uniquely identify the user within the system.

The User class contains functions such as `editCode()`, `joinSession()`, and `viewOutput()`, which allow the user to interact with the collaborative editor. Through these methods, users can write code, participate in collaborative sessions, and view the output generated by the system. The User class is associated with the Session class, indicating that multiple users can participate in coding sessions.

2. Session Class :

The Session class represents collaborative coding sessions where multiple users work together on the same code. It contains attributes such as `sessionId`, `codeContent`, and `activeUsers`, which store information about the current coding session and the users participating in it.

The Session class includes methods such as `synchronizeCode()`, `sendExecutionRequest()`, and `sendAIQuery()`. These methods allow the system to synchronize code updates among participants, send code for execution, and request assistance from the AI chatbot. The Session class acts as an intermediary between the users and other system components.

3. CodeEditor Class :

The CodeEditor class represents the main collaborative coding environment. It manages coding sessions and coordinates communication between different system components.

This class contains attributes such as `editorId`, `editorName`, and `activeSessions`, which store information about the editor instance and active collaborative sessions. The methods included in this class are `manageSession()`, `synchronizeCode()`, `handleAIQuery()`, and `sendToExecutionEngine()`. These methods enable the editor to manage collaborative sessions, process AI queries, and forward code execution requests to the exe-

cution engine.

4. AIChatbot class :

The AIChatbot class provides intelligent coding assistance to users. It helps users by analyzing code and providing debugging suggestions and improvements.

The class contains attributes such as botId and botName, which identify the AI assistant. It also includes methods such as processQuery(), debugHelp(), and suggestCode(), which allow the chatbot to process user queries, provide debugging assistance, and recommend improvements to the code.

The AIChatbot interacts with the Session and CodeEditor classes to receive queries from users and deliver AI-generated suggestions.

5. Course Class :

The Course class represents educational content and learning resources available within the system. It stores information related to programming courses and coding sessions.

This class includes attributes such as courseId, courseName, and content, which define the course information and materials. It also includes methods such as uploadMaterials() and manageSessions(), which allow instructors to manage course resources and organize coding sessions for students.

The Course class is associated with the CodeEditor class, indicating that coding sessions can be linked to specific courses.

6. CodeExecutionEngine Class :

The CodeExecutionEngine class is responsible for compiling and executing the code written by users. It processes code execution requests and returns the output or error messages generated during execution.

The class includes attributes such as engineId, maxExecTime, and sessionLogs, which store execution-related information. It also contains methods such as executeCode() and generateOutput(), which run the program and produce the corresponding output.

This class works closely with the CodeEditor class to execute user code and return the results to the collaborative editor.

Overall, the Class Diagram provides a clear representation of the struc-

tural design of the Real-Time Collaborative Code Editor System. It shows how different classes interact with each other to support collaborative coding, AI-based assistance, course management, and code execution. The relationships between these classes help define the architecture of the system and guide the implementation of the software components.

5.2 Entity-Relationship Diagram (ERD)

This figure 5.2 represents the Entity-Relationship (ER) Diagram of the Real-time Collaborative Code Editor system. An ER diagram is used to describe the structure of a database by illustrating entities, their attributes, and the relationships between them. It helps in understanding how data is stored, organized, and accessed within the system.

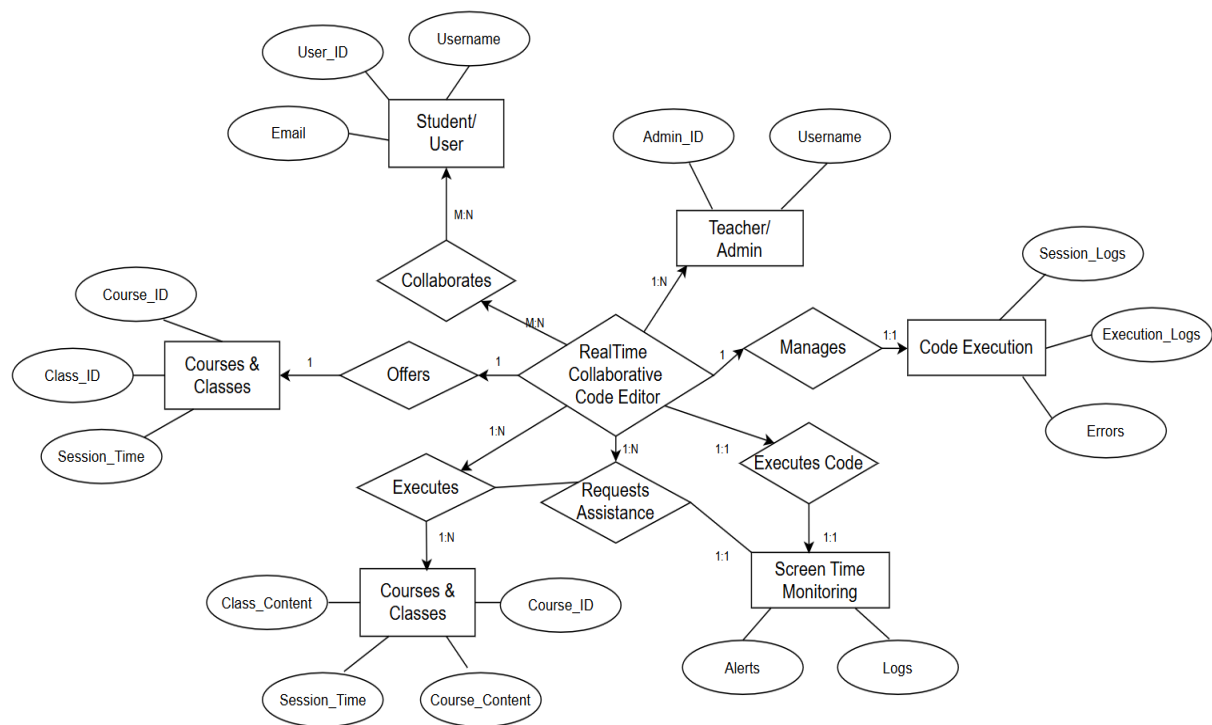


Figure 5.2: ER Diagram

The ER diagram includes important entities such as Student/User, Teacher/Admin, Courses & Classes, Code Execution, Screen Time Monitoring, and the Real-Time Collaborative Code Editor. These entities interact with each other through various relationships such as collaboration, course management, code execution, and AI assistance.

1. Student / User Entity :

The Student/User entity represents individuals who use the collaborative coding platform to write, edit, and execute code. Each student has attributes such as User_ID, Username, and Email, which uniquely identify the user and store their login and contact information.

Students interact with the Real-Time Collaborative Code Editor to par-

ticipate in coding sessions. The relationship “Collaborates” represents the interaction between users and the editor, allowing multiple users to work on the same coding project simultaneously. This relationship follows a many-to-many (M:N) relationship because multiple students can collaborate in multiple coding sessions.

2. Teacher / Admin Entity :

The Teacher/Admin entity represents instructors or administrators who manage the coding platform. This entity contains attributes such as Admin_ID and Username, which uniquely identify each instructor.

Teachers interact with the Real-Time Collaborative Code Editor through the relationship “Manages”, which allows them to control coding sessions, manage courses, and monitor student activities. This relationship typically follows a one-to-many (1:N) relationship because one instructor can manage multiple coding sessions or classes.

3. Courses & Classes Entity :

The Courses & Classes entity stores information related to educational content and coding sessions conducted within the platform. It includes attributes such as Course_ID, Class_ID, and Session_Time, which define different courses and class schedules.

The relationship “Offers” connects the Real-Time Collaborative Code Editor with the Courses & Classes entity. Through this relationship, the system provides coding sessions and learning materials associated with specific courses.

Another relationship called “Executes” represents how courses are used within collaborative coding sessions where students work on coding exercises and projects.

4. Real-Time Collaborative Code Editor Entity :

The Real-Time Collaborative Code Editor is the central entity in the ER diagram. It connects various system components and manages the collaborative coding environment.

This entity coordinates interactions between students, teachers, course modules, and the execution engine. It also handles relationships such as collaboration between users, course management by instructors, and code execution requests.

The collaborative editor ensures that all participants in a coding session

can view and edit the same code in real time.

5. Code Execution Entity :

The Code Execution entity represents the component responsible for running and compiling code written by users. It includes attributes such as Session_Logs, Execution_Logs, and Errors, which store information about program execution.

The relationship “Executes Code” connects the collaborative editor with the code execution module. When users submit code for execution, the system processes the request and returns the output or error messages. These execution details are stored in logs for monitoring and debugging purposes.

6. Screen Time Monitoring Entity :

The Screen Time Monitoring entity tracks how long students use the coding platform. It helps ensure responsible usage of the system and prevents excessive screen time.

This entity contains attributes such as Alerts and Logs, which record user activity and generate warnings when screen time exceeds predefined limits. The relationship between the collaborative editor and screen time monitoring ensures that user activity is continuously tracked.

Overall, the ER diagram provides a structured representation of the database design for the Real-Time Collaborative Code Editor System. It shows how different entities such as users, teachers, courses, execution modules, and monitoring systems interact with each other. The relationships between these entities help manage collaborative coding sessions, execute programs, monitor activity, and organize course content within the platform.

5.3 Activity Diagram

5.3 illustrates the Activity Diagram of the Real-time Collaborative Code Editor system. An activity diagram is used to represent the workflow of a system by showing the sequence of activities and decisions that occur during the execution of a process. It helps in understanding how different operations are performed step by step within the system.

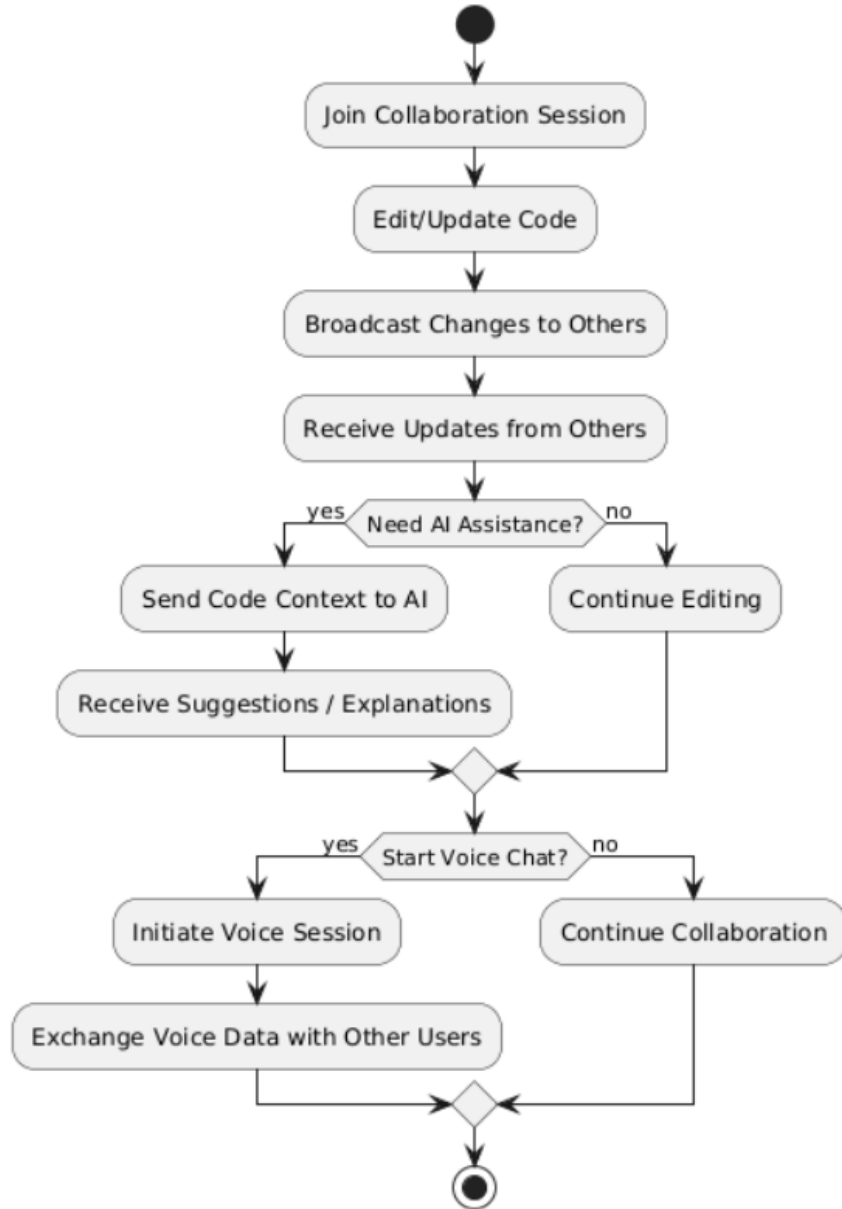


Figure 5.3: Activity Diagram

In the proposed system, the activity diagram describes how users interact with the collaborative coding platform, how code updates are synchronized among participants, and how additional features such as AI

assistance and voice communication support the coding process.

1. Join Collaboration Session:

The workflow begins when a user joins a collaboration session. A session represents a shared coding environment where multiple participants can work on the same code simultaneously. Users can either create a new session or join an existing one using a session identifier.

Once the session is established, the user gains access to the shared code editor and becomes part of the collaborative workspace.

2. Edit Code:

After joining the session, the user can start writing or modifying code in the collaborative editor. This step represents the primary activity in the system where users contribute to the development of the program.

The editor records each modification made by the user, including typing, deleting, or updating code segments.

3. Broadcast Changes to Others:

When a user edits the code, the system automatically broadcasts the changes to all connected participants. This ensures that every collaborator in the session receives the updated version of the code in real time.

The broadcasting process is handled by the synchronization mechanism of the system.

4. Receive Updates from Others:

In addition to sending updates, the user also receives code changes made by other collaborators. These updates are applied to the local editor so that all participants view the same code simultaneously.

This step ensures that the collaborative environment remains synchronized and prevents inconsistencies in the shared codebase.

5. Need AI Assistance:

At this stage, the system checks whether the user requires assistance from the AI module. If the user does not request AI help, the workflow continues with normal code editing activities.

If assistance is required, the system proceeds to the next step where the code context is sent to the AI module for analysis.

6. Send Code Context to AI:

When AI assistance is requested, the system sends the current code context to the AI module. This allows the AI system to analyze the program and identify possible improvements or errors.

The AI module processes the code using machine learning models or predefined programming rules.

7. Receive Suggestions:

After analyzing the code, the AI module returns suggestions, debugging hints, or explanations to the user. These suggestions help improve code quality and assist users in resolving programming errors.

Users can review the suggestions and apply them if necessary.

8. Start Voice Chat:

The system then checks whether the user wants to initiate voice communication with other collaborators. If voice chat is not required, the workflow continues with collaborative coding.

If the user chooses to start voice communication, the system proceeds to establish a voice session.

9. Initiate Voice Session:

In this step, the system initiates a voice communication session among collaborators. This allows participants to discuss coding problems, share ideas, and coordinate their tasks more effectively. The voice session is linked to the collaboration session so that only participants in the same workspace can communicate.

10. Exchange Voice Data with Other Users:

Once the voice session is active, users can send and receive voice data in real time. The system transmits audio streams between participants, enabling smooth verbal communication during coding activities.

This feature improves collaboration and allows faster problem-solving among team members.

11. Continue Collaboration:

After completing these activities, users continue working on the col-

laborative coding session. They can keep editing code, receiving updates, requesting AI assistance, or communicating with other participants until the coding task is completed.

Overall, the Activity Diagram represents the complete workflow of the collaborative coding system, illustrating how users interact with the editor, how updates are synchronized, and how AI assistance and voice communication enhance the collaborative experience.

5.4 Sequence Diagram

The following diagram 5.4 presents the Sequence Diagram of the Real-time Collaborative Code Editor system. A sequence diagram is a type of UML diagram that illustrates how different components of a system interact with each other over time. It shows the order of messages exchanged between objects and helps in understanding the dynamic behavior of the system during execution.

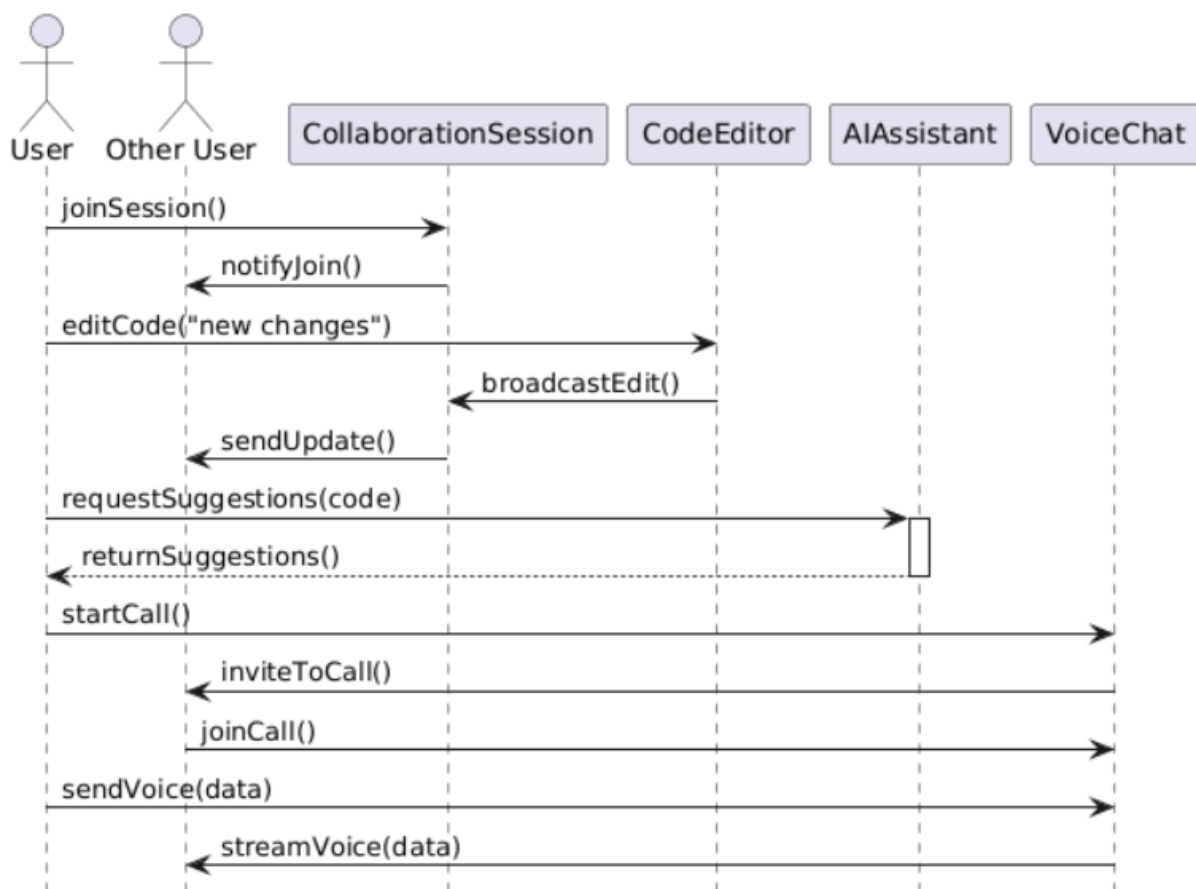


Figure 5.4: Sequence Diagram

In the proposed system, the sequence diagram describes the interaction between the User, Code Editor, Collaboration Session Manager, AI Assistant, and Voice Chat module during a collaborative coding session. The diagram highlights the flow of events from the moment a user joins a session to the point where collaborative coding, AI assistance, and communication take place.

1. Join Session:

The sequence begins when a user joins a collaborative coding session.

The user sends a request to the system to connect to an existing session or create a new one. The collaboration session manager validates the request and establishes the session.

Once the session is successfully created or joined, the system notifies other participants that a new user has joined the collaborative workspace.

2. Code Editing and Updates:

After joining the session, the user begins editing code within the collaborative editor. The code editor records the changes made by the user and sends the updates to the collaboration session manager.

The collaboration module processes these edits and ensures that the updated code is synchronized across all connected users. This allows multiple participants to view and modify the same code simultaneously.

3. Synchronization of Code Changes:

When the collaboration session manager receives the updated code, it broadcasts the changes to other participants in the session. Each connected user receives the updated code and the editor automatically integrates the changes.

This process ensures that all users are working with the same version of the code and that updates are reflected in real time.

4. AI Suggestions:

At any point during the coding session, the user may request assistance from the AI Assistant module. The code editor sends the current code context to the AI system for analysis.

The AI assistant examines the code and generates suggestions such as identifying syntax errors, recommending improvements, or explaining code segments. These suggestions are then returned to the user and displayed within the editor interface.

This feature helps users improve their code quality and understand programming concepts more effectively.

5. Voice Call Initialization:

To facilitate better collaboration, users may initiate a voice communication session. When a user starts a voice call, the system sends an invitation to other participants in the collaboration session.

Participants can accept the invitation and join the voice communication channel. This allows team members to discuss coding tasks and coordinate their work.

6. Voice Streaming:

Once the voice session is established, users can exchange voice data in real time. Audio streams are transmitted through the voice chat module and delivered to all connected participants.

This real-time communication helps collaborators discuss problems, share ideas, and resolve coding issues more efficiently.

7. Result:

The sequence diagram demonstrates how different components of the system interact to support collaborative coding. Through the integration of code editing, synchronization, AI assistance, and voice communication, the system provides a comprehensive environment for real-time collaborative programming.

Overall, the sequence diagram highlights the dynamic interactions that enable users to collaborate effectively within the coding platform.

Chapter 6

Results and Discussion

This section presents the results obtained after implementing the Real-Time Collaborative Code Editor System. The developed system was tested to evaluate its ability to support collaborative coding, real-time communication, AI-based assistance, and multi-language code execution. The platform is designed to provide an easy-to-use interface so that students can participate in coding sessions without requiring complex installations or technical setup.

The following figures illustrate the working of the system and demonstrate how different components interact to provide a complete collaborative coding environment.

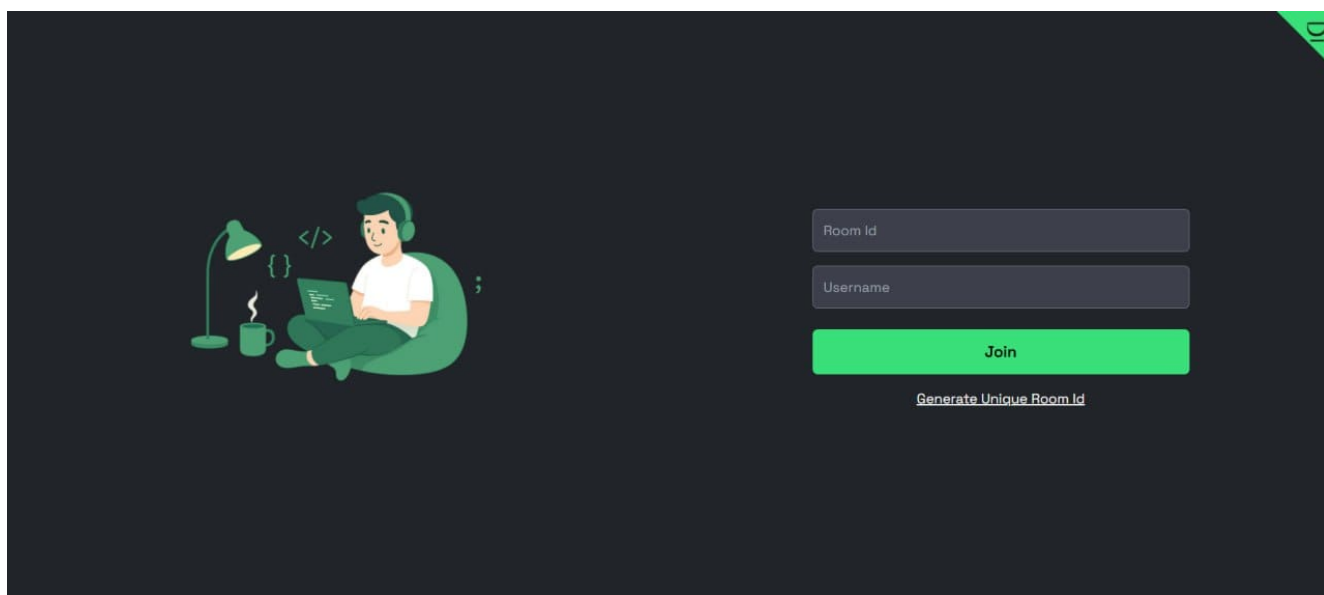


Figure 6.1: Home page

In figure:6.1 , The home page acts as the entry point of the collaborative coding platform. It provides a simple interface where users can enter their

Room ID and Username to join an existing collaboration session. This feature allows multiple users to connect to the same coding environment and work together in real time.

In addition to joining existing sessions, the system also provides an option to generate a unique room ID, which enables users to create a new collaborative session. The generated room ID can then be shared with other participants who wish to join the same workspace. This approach simplifies the process of collaboration and removes the need for complex user authentication or account creation.

The minimal and user-friendly design of the home page ensures that even beginners can quickly access the coding environment without confusion.

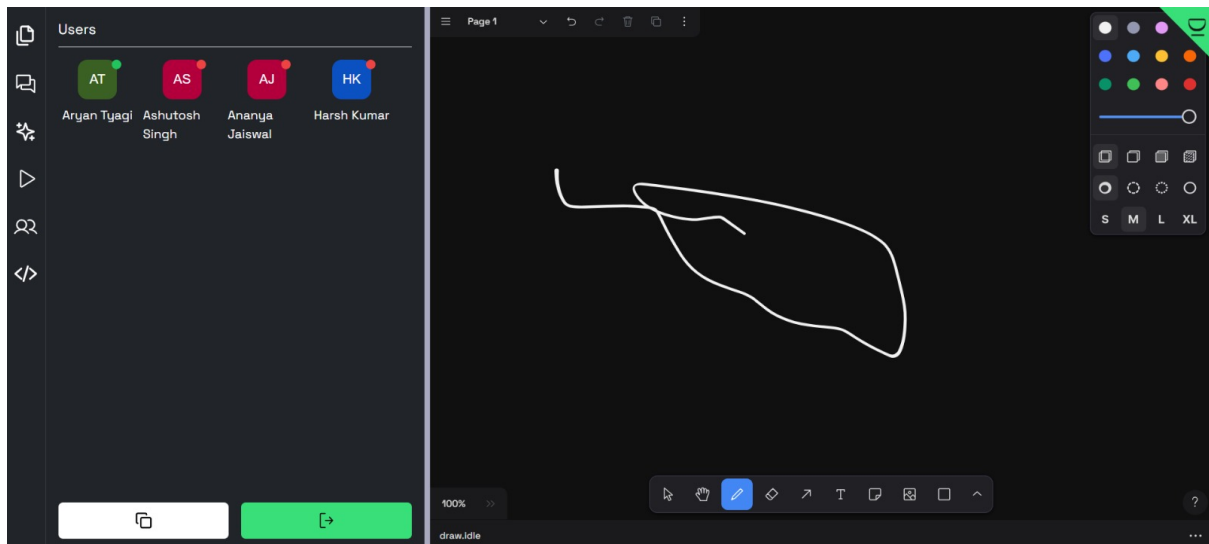


Figure 6.2: Shared Canvas for Visual Explanation

In figure:6.2 , The collaborative canvas is designed to support visual explanation during coding sessions. Teachers or participants can use this canvas to draw diagrams, explain logic structures, or demonstrate algorithmic flow. This feature is particularly useful during teaching sessions where instructors need to illustrate concepts in a visual manner.

The drawing tools allow users to sketch shapes, highlight important sections, and explain programming logic interactively. Students can follow the explanation in real time while the instructor demonstrates concepts on the shared canvas.

This functionality enhances the learning experience by combining coding with visual explanations, making complex programming concepts eas-

ier to understand.

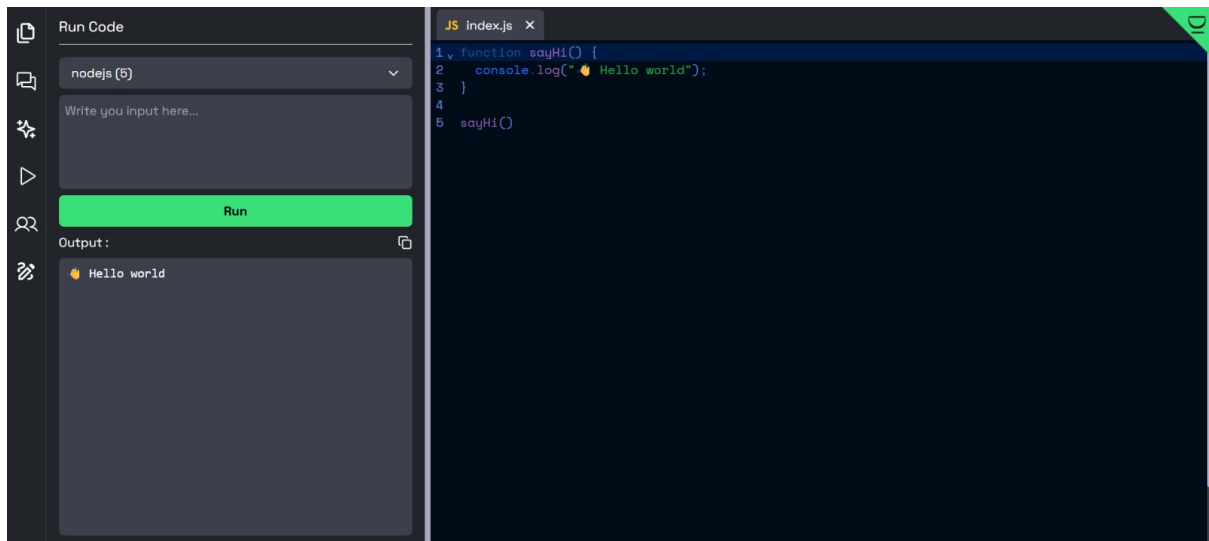


Figure 6.3: Code Execution and Output Generation

In figure:6.3 , The system includes an integrated code execution engine that allows users to compile and run programs directly within the collaborative editor. Users can write their code in the editor, select the programming language, and execute the program using the built-in execution module.

Once the program is executed, the system displays the generated output in the output panel. If the code contains errors, the system provides appropriate error messages to help users identify and fix issues. This functionality allows users to test and debug their code without leaving the collaborative environment.

By integrating the code execution engine into the platform, the system eliminates the need for installing external compilers or development tools, which is especially beneficial for students working on shared laboratory computers.

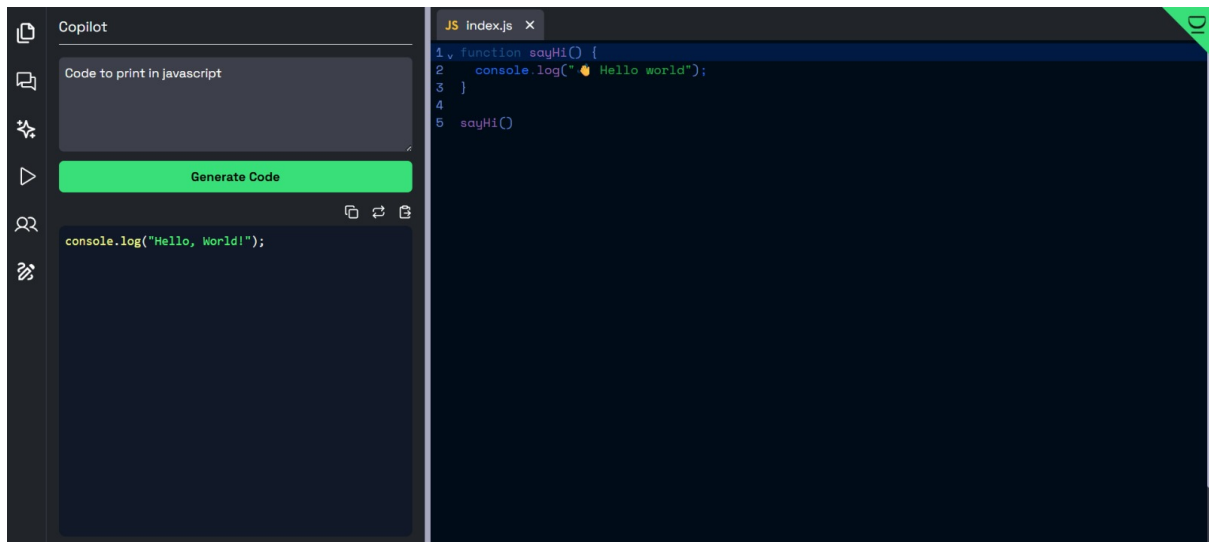


Figure 6.4: AI Code Generation using Copilot

In figure:6.4 , The AI assistance module provides intelligent support to users while they are writing code. Users can enter prompts or coding requests in the AI interface, and the system generates relevant code snippets or suggestions using an AI-based model.

This feature helps users quickly generate code for common programming tasks and provides guidance when they encounter difficulties during coding. The AI module can also assist beginners by suggesting solutions, correcting syntax errors, and recommending improved coding practices.

By integrating AI support into the collaborative editor, the system enhances productivity and encourages users to explore programming concepts more effectively.

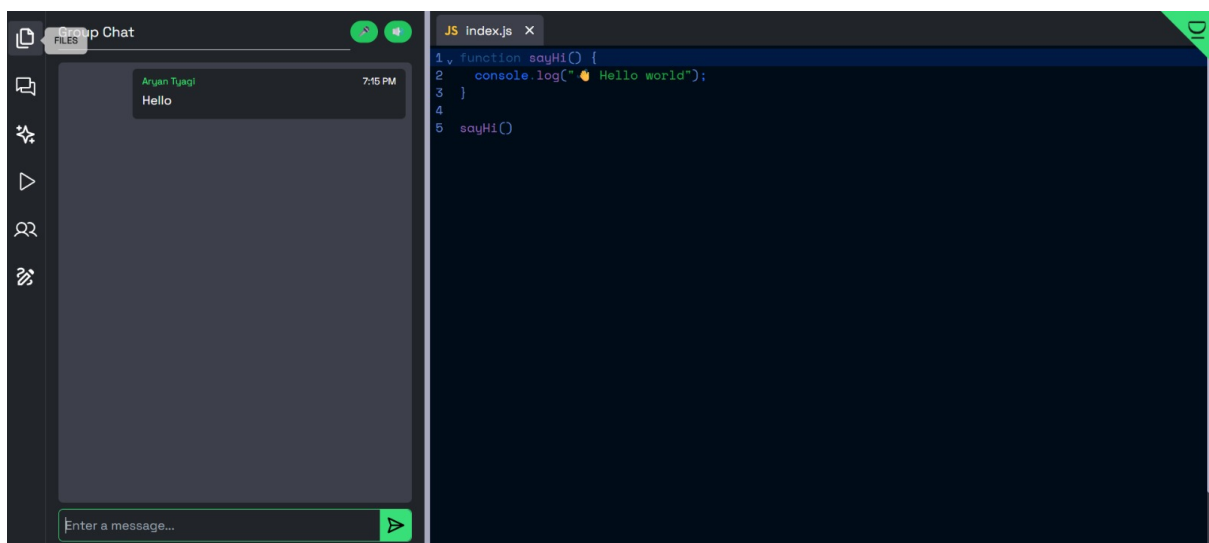


Figure 6.5: Chat Interface for Collaboration

In figure:6.5 , The platform includes a built-in chat system that enables communication between participants during collaborative coding sessions. Users can send messages to each other while working on the same coding project, allowing them to discuss problems and coordinate their tasks.

The chat feature is particularly useful when multiple users are working together on a project, as it allows them to exchange ideas, clarify doubts, and share solutions in real time. This communication capability improves teamwork and makes the collaborative coding process more efficient.

By combining messaging with real-time code editing, the platform creates a more interactive and engaging development environment.

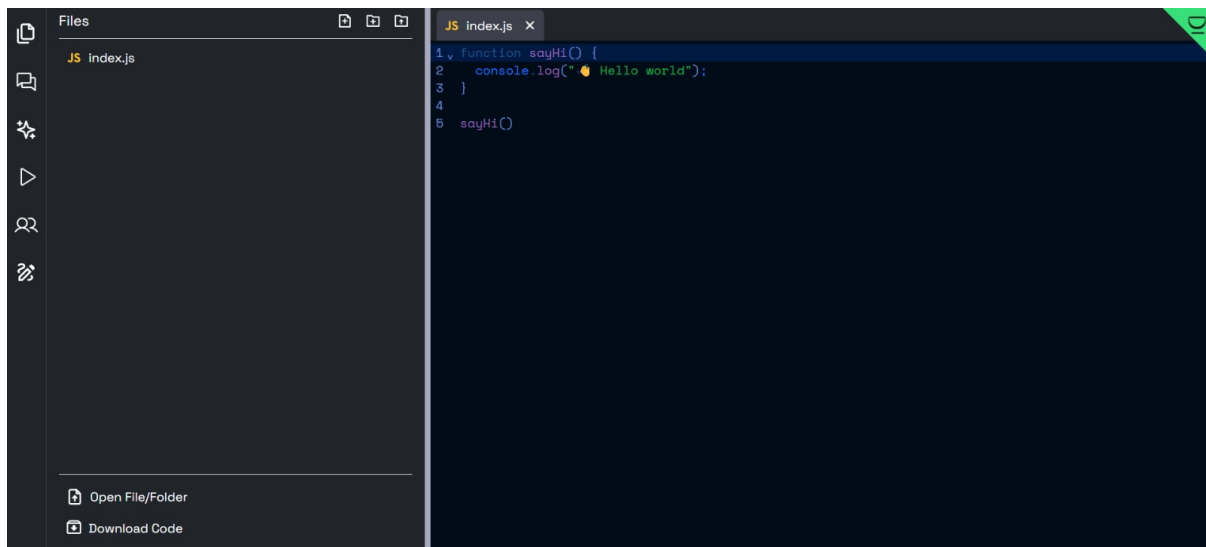


Figure 6.6: Multi-language Code Editing Interface

In figure:6.6 , The collaborative editor supports multiple programming languages, allowing users to write and execute code in different languages within the same platform. This flexibility enables students to practice coding in languages such as JavaScript, Python, or other supported languages without needing separate development tools.

The editor provides features such as syntax highlighting, structured code formatting, and easy code execution. These features improve readability and help users write better structured programs.

Supporting multiple programming languages makes the platform suitable for educational environments where students need to learn and experiment with different programming technologies.

The results demonstrate that the developed system successfully provides a complete collaborative coding environment that integrates coding, communication, and AI assistance into a single platform. The user interface is designed to be simple and intuitive, making it accessible to students who may not have advanced technical knowledge.

Features such as the collaborative canvas, chat communication, AI code assistance, and multi-language code execution enhance the overall usability of the platform. These functionalities enable users to collaborate effectively, learn programming concepts interactively, and improve their coding skills in a supportive environment.

Overall, the implementation of the Real-Time Collaborative Code Editor demonstrates how modern technologies can be used to improve programming education and provide students with real-world collaborative development experience.

Chapter 7

Future Work

1. Concept Development :

The first step in further development is to clearly define the core concept of the “Code & Chill” platform. This involves identifying and finalizing the key features that will make the platform useful and engaging for students and developers. Important functionalities such as real-time collaborative coding, AI-assisted programming guidance, and a simple student-friendly interface need to be carefully designed. To make the platform more practical and effective, surveys and feedback sessions can be conducted with students and teachers to understand their learning challenges and expectations from such a system. The insights collected from these discussions will help refine the project vision and ensure that the platform truly supports collaborative learning and programming practice.

2. UI/UX Design :

The user interface and user experience design play a crucial role in the success of the platform. The goal is to create an intuitive and visually appealing design that allows users to easily navigate through the system without confusion. This process involves creating wireframes, interactive prototypes, and clear navigation flows that guide users smoothly through coding sessions, collaborative projects, and learning resources. The interface should be simple enough for beginners while still offering useful tools for more experienced users. By focusing on clarity, accessibility, and simplicity, the platform can ensure that students of different age groups can easily participate in coding activities and collaborative learning.

3. Backend Data Setup :

A strong backend infrastructure is necessary to support the real-time collaborative features of the platform. The backend system will manage multiple coding sessions simultaneously while ensuring that all partici-

pants see synchronized updates of the code. It will also handle secure storage of project files, user authentication, and communication between participants through chat or collaborative editing tools. Efficient server-side architecture and database management will be implemented to ensure that the system remains stable, secure, and scalable even when many users are working together at the same time.

4. AI Integration :

Artificial intelligence can significantly improve the coding experience for students by providing intelligent assistance during programming. The platform will integrate an AI-powered chatbot that can guide users while writing code, suggest improvements, and answer programming-related questions. The AI system can analyze the context of the code and provide helpful hints, syntax suggestions, or debugging assistance. Over time, the AI module can be enhanced by analyzing user interactions and feedback, allowing it to provide more accurate suggestions and better learning support for students.

5. Testing and Feedback :

Before fully deploying the platform, extensive testing must be conducted to ensure that all features work correctly and efficiently. This includes testing real-time collaborative editing, voice or text chat communication, AI suggestions, and code execution functionalities. Different scenarios will be tested to identify possible bugs, synchronization delays, or usability issues. Feedback from students and teachers will also be collected during the testing phase to understand how the platform performs in real learning environments. This feedback will help developers improve system performance, usability, and overall user experience.

6. Enhancements :

In the future, additional features can be introduced to further improve the platform and make it more engaging for students. These enhancements may include integrating video tutorials to explain programming concepts, providing coding problem sets for practice during holidays, and offering ready-made project templates for collaborative work. The system can also include notification features that remind users about screen time limits and track achievements to encourage healthy coding habits. Such enhancements will transform the platform into a more comprehensive and

interactive learning environment for programming.

Bibliography

- [1] Grover, S., & Pea, R. Computational thinking in K–12: A review of the state of the field. *Educational Researcher*, 47(3), 166–186, 2018.
- [2] Sentance, S., & Waite, J. Teaching computer programming in schools: Pedagogical approaches and challenges. *International Journal of Computer Science Education in Schools*, 1(1), 1–12, 2017.
- [3] Wing, J. Computational thinking as an educational goal. *Communications of the ACM*, 60(10), 28–30, 2017.
- [4] Luxton-Reilly, A., et al. The influence of collaboration in programming education. *Proceedings of the ACM Technical Symposium on Computer Science Education*, 1054–1060, 2018.
- [5] Lye, S. Y., & Koh, J. H. L. Review of learning approaches in programming education. *Educational Technology*, 56(5), 44–52, 2016.
- [6] Replit. Multiplayer coding platform overview. Available at: <https://replit.com/site/multiplayer>, 2021.
- [7] Microsoft. Visual Studio Live Share Documentation. Available at: <https://learn.microsoft.com/en-us/visualstudio/liveshare/>, 2020.
- [8] Kasneci, E., et al. ChatGPT and education: Opportunities and challenges. *British Journal of Educational Technology*, 2023.
- [9] Vaithilingam, P., et al. Expectations and experiences of AI code assistants. *CHI Conference on Human Factors in Computing Systems*, 2022.
- [10] Livingstone, S., & Third, A. Children and digital media: Impacts and ethical design. *UNICEF Office of Research*, 2017.
- [11] Kafai, Y., & Burke, Q. Connected Code: Why Children Need to Learn Programming. MIT Press, 2016.

- [12] Resnick, M., et al. Scratch: Programming for all. *Communications of the ACM*, 60(11), 60–67, 2017.
- [13] Fagerlund, J., et al. Artificial Intelligence in programming education: A systematic review. *Computers and Education: Artificial Intelligence*, 2023.
- [14] Bennedsen, J., et al. Novice programming students and the role of collaborative work. *ITiCSE Conference Proceedings*, 2019.
- [15] Shute, V., Sun, C., & Clarke, J. Demystifying computational thinking. *Educational Research Review*, 22, 142–158, 2017.
- [16] Guzdial, M. Computing education as a foundation for 21st century skills. *Communications of the ACM*, 2019.
- [17] Hong Kong Education Bureau. IT in Education Policy: Promoting Computational Thinking. Available at: <https://www.edb.gov.hk>, 2019.
- [18] Sentance, S., et al. Equity in computing education: Understanding teacher and student needs. *ACM Transactions on Computing Education*, 2019.
- [19] Balik, M., et al. Comparative study of online IDEs for education. *Journal of Computing in Higher Education*, 2018.
- [20] Wong, G., et al. School perceptions of coding education in K–12: Large scale study. *IEEE TALE Conference Proceedings*, 2017.
- [21] Yadav, A., et al. Computational thinking in teacher education. *ACM Transactions on Computing Education*, 2016.

Appendix A

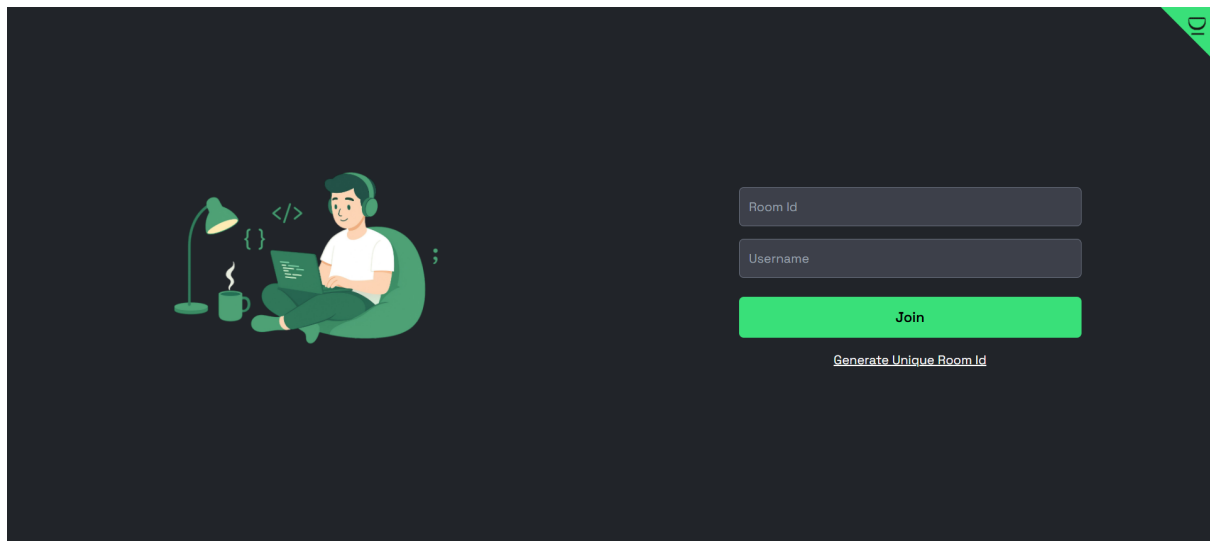


Figure 7.1: Screenshot Of Application

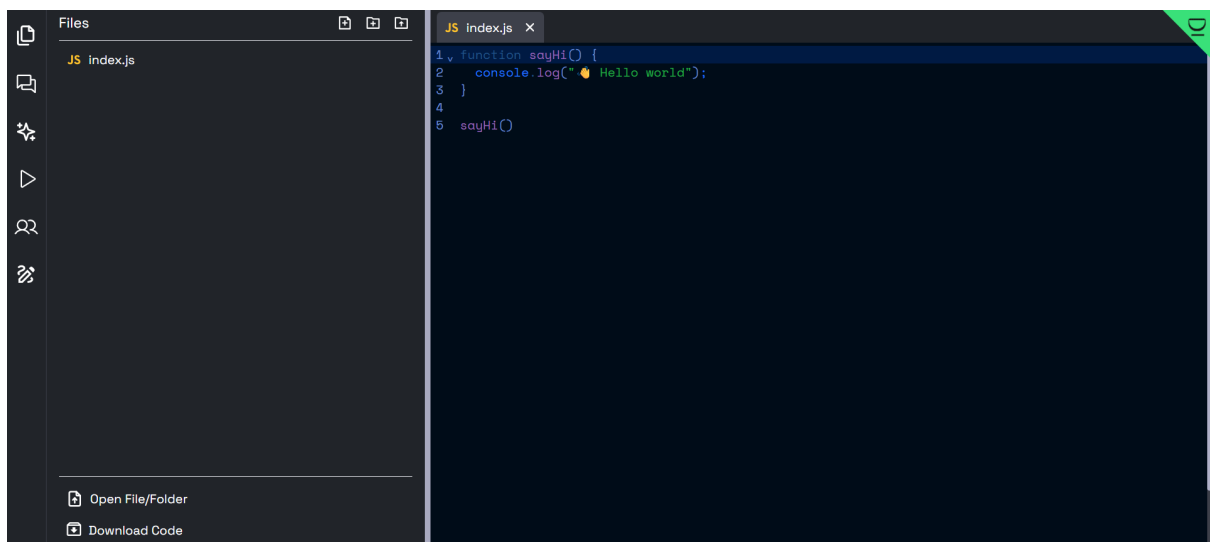


Figure 7.2: Screenshot Of Application

Appendix B

Enhancing Computational Literacy: An Accessible Web-Based Collaborative Code Editor for Schools

Aryan Tyagi Ashutosh Singh Ananya Jaiswal
Ajay Kumar Garg Engineering College Ajay Kumar Garg Engineering College Ajay Kumar Garg Engineering College
Ghaziabad, Uttar Pradesh Ghaziabad, Uttar Pradesh Ghaziabad, Uttar Pradesh
aryan2213184@akgec.ac.in ashutosh2213185@akgec.ac.in ananya2213085@akgec.ac.in

Harsh Kumar Mr. Nikhil Kumar
Ajay Kumar Garg Engineering College Ajay Kumar Garg Engineering College
Ghaziabad, Uttar Pradesh Ghaziabad, Uttar Pradesh
harsh2213190@akgec.ac.in nikhilcs33@gmail.com

Abstract—Coding education is now a priority worldwide, but most school systems find themselves unable to give students substantial exposure to the actual ways of programming in the real world. Syntax drills are what typical curricula focus on, as opposed to project-based learning in teams, with school laboratories crippled by antiquated equipment, pricey software, and high learning curves. This article suggests designing and creating an online collaborative code editor that operates in real-time especially for students in school, covering both technical and pedagogical needs. The tool allows students to code simultaneously synchronously using a web browser without the need for installs or compulsory accounts, incorporates an AI assistant to provide context-related advice, and uses screen-time notifications to promote healthy digital lifestyles. We survey the latest studies on collaborative editing platforms, K-12 computational thinking education, cloud-based Integrated Development Environments (IDEs), and AI-powered tutoring systems. We design an architecture based on CRDTs and light-weight web technologies with emphasis on accessible UI/UX for beginner learners. The suggested answer is bound to enhance participation, cooperation, and inclusion in coding instruction, while bringing to the forefront new challenges in teacher preparation, fairness, and ethical use of AI.

Index Terms—Collaborative Code Editor, Artificial intelligence, Chat Bot, Natural Language Processing

I. INTRODUCTION

Over the past decade, computational literacy has been recognized as a critical skill in the digital era. As artificial intelligence, data science, and automation expand across industries, schools face pressure to prepare students with programming knowledge that goes beyond theory into practice. Although countries have introduced Python, Java, and Scratch into secondary curricula, students' real exposure to coding remains limited [1]. Low-resource schools in particular struggle with outdated labs, limited personal devices, undertrained instructors, and install-heavy software with steep learning curves [2].

Coding education has mostly been conceptual because of poor infrastructure, inaccessibility to personal laptops and

computers, and due to widespread socio-economic differences. To bridge this gap, we have introduced our collaborative code editor [5].

A trend has been observed that students often grow up with collaborative technologies like instant messaging and various others, but it rarely extends to programming education. Whereas simple IDE's such as Scratch or Code.org simplify code but lack real-world depth, and professional tools such as Replit or VS Code are often very difficult to navigate through via for novices or students who are studying in school. [8].

A clear yet concise solution comes to our mind. We can create a browser-based code editor that requires no installation or registration, and which allows students to debug code in real-time, and also helps teachers to communicate through AI tutoring adds further value and AI ChatBot will also be installed in it for further assistance [9]. While it is not a replacement for teachers, this kind of tool can surely extend classroom support and further the students' interest and innovation in coding [11].

Despite an increase in the computational literacy of our country, school instruction remains fragmented. It is often observed that teachers rarely have any real and practical coding experience, thereby not being able to convey the concepts of coding fully. This often derails the students' interest in coding, thereby derailing innovation in tech. Different socio-economic backgrounds also derails this innovation as students with considerable resources often have an edge over their disadvantaged peers [4][5].

Even when tools exist, usability is a challenge. Professional IDEs intimidate beginners [6], cloud tools assume knowledge of advanced workflows [7], and block-based systems fail to transition students smoothly to text-based programming [8]. Beyond tools, the absence of collaborative learning and modern pedagogy reduces motivation and problem-solving gains [9][10].

Figure 7.3: Research paper

Appendix C

SDG 4: Quality Education

The project enhances digital literacy by giving students real-time coding practice, AI chatbot support, and collaborative tools. It directly contributes to Target 4.4: increasing technical and vocational skills relevant for future jobs.

SDG 9: Industry, Innovation, and Infrastructure

By integrating AI and machine learning with education, the project fosters innovation and scalable infrastructure for digital learning. This supports Target 9.5: strengthening scientific research and advancing technology development.

SDG 10 - Reduced Inequalities

The platform reduces education gaps by offering affordable, browser-based coding access to underserved regions with limited infrastructure. It addresses Target 10.2: empowering and promoting inclusion of all learners.

SDG 17: Partnerships for the Goals The project builds collaboration between students, teachers, researchers, and institutions to promote shared learning and innovation. This fulfills Target 17.6: enhancing global partnerships for technology and knowledge exchange.

Certificate of Compliance with United Nations Sustainable Development Goals

This is to certify that the project titled **Real-time Collaborative Code Editor**, submitted by **Aryan Tyagi (2200270130031)**, **Ananya Jaiswal (2200270130017)**, **Ashutosh Singh (2200270130035)**, **Harsh Kumar (2200270130062)**, final year students of the Bachelor of Technology in Information Technology program at Ajay Kumar Garg Engineering College, Ghaziabad, have been reviewed and found to be in alignment with the following United Nations Sustainable Development Goals (SDGs). All efforts have been made to the best of our ability and knowledge that no other SDGs are compromised or negatively impacted.

SDG No.	SDG Name	Relevance	SDG No.	SDG Name	Relevance
1	No Poverty	☐	10	Reduced Inequalities	☐
2	Zero Hunger	☐	11	Sustainable Cities and Communities	☐
3	Good Health and Well-being	☐	12	Responsible Consumption and Production	☐
4	Quality Education	☐	13	Climate Action	☐
5	Gender Equality	☐	14	Life Below Water	☐
6	Clean Water and Sanitation	☐	15	Life on Land	☐
7	Affordable and Clean Energy	☐	16	Peace, Justice, and Strong Institutions	☐
8	Decent Work and Economic Growth	☐	17	Partnerships for the Goals	☐
9	Industry, Innovation, and Infrastructure	☐			

Signature of the Students

Aryan Tyagi (2200270130031)

Ananya Jaiswal (2200270130017)

Ashutosh Singh (2200270130035)

Harsh Kumar (2200270130062)

Signature of the Supervisor

Mr. Nikhil Kumar

Figure 7.4: SDG

Appendix D

Figure 5.8 shows the plagiarism report of our project, which is 8 percentage

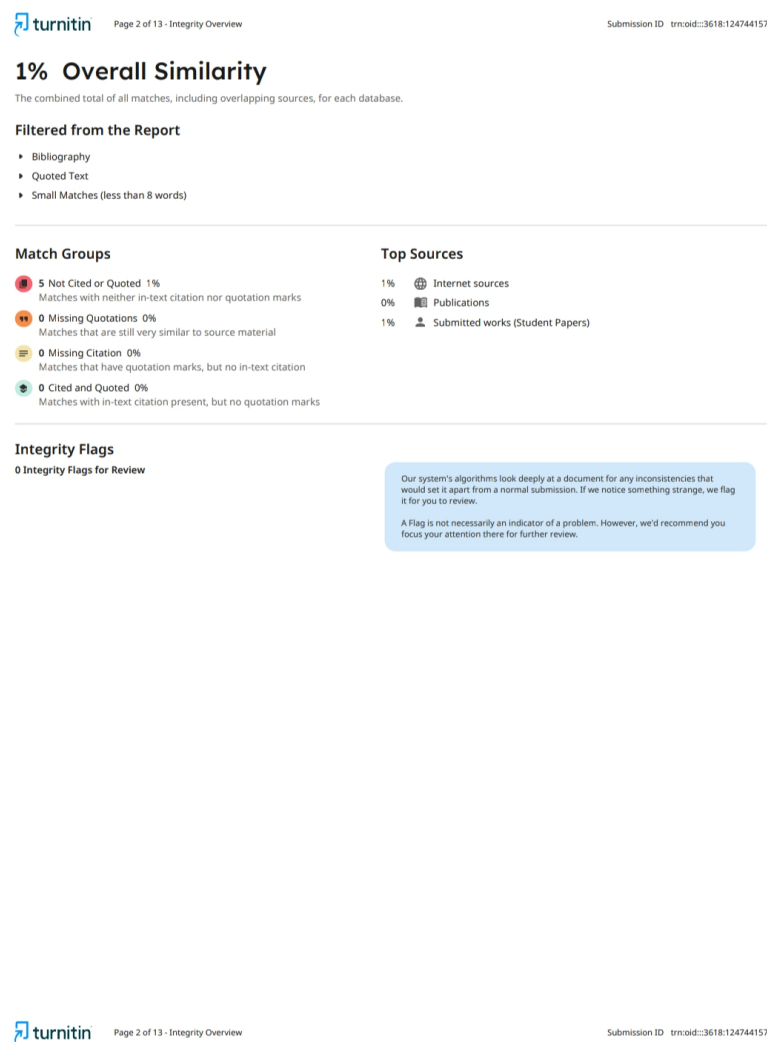


Figure 7.5: Plagiarism Report

Figure 5.9 shows the AI report of our project, which is less than 20 percentage

Page 2 of 12 - AI Writing OverviewSubmission ID trn:oid::3618:124744157

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Page 2 of 12 - AI Writing OverviewSubmission ID trn:oid::3618:124744157

Figure 7.6: AI Report